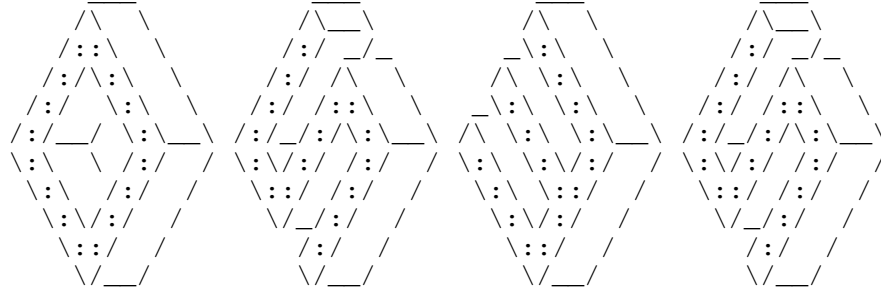


# Fine-grained role-based access control by encryption

Andreas Severin Hauch Trøstrup  
Computer Science  
IT University of Copenhagen  
[atro@itu.dk](mailto:atro@itu.dk)

Lucas Frey Torres Hanson  
Computer Science  
IT University of Copenhagen  
[luha@itu.dk](mailto:luha@itu.dk)



**Abstract**—Data lakes supporting multiple query engines lack fine-grained access control; only “fully managed all-in-one cloud platforms” like Snowflake or Databricks offer it, within their own ecosystem.

We propose Object Store Wrapper Service, a system enforcing column-level role-based access control at the Object Store layer using Parquet Modular Encryption with envelope encryption, and still supports query engines that are S3 compatible.

Our evaluation shows that OSWS adds an acceptable overhead for tiny to small Parquet files, when using a DEK cache, with warm p95 GET latency of 30ms for a 5MB Parquet file, but with other design choices a more efficient solution is possible.

OSWS demonstrates that encryption-based access control at the Object Store layer is a viable approach for data lakes, though limitations exist for query engines that cache Parquet metadata.

## TODO LIST

- ☐ In **methodology**, add info about setup for testing DuckLake and what the issues were
- ☐ System Design has to be refactored to not mention code files
- ☐ In **Discussion**, we have to add more info about why Parquet Sharp is not the way forward, as it does most work sequentially, and our other idea would allow for pre-fetching the keys and doing work asynchronously
- ☒ Why is footer encryption not enabled.
- ☒ Fix the not-defined times for how slow it is
- ☒ Lucas uses p99 for micro stuff. Use p95 instead to match e2e
- ☐ Include figure for parquet with unencrypted footer in appendix
- ☐ LRU has to be mentioned in system design
- ☐ Seems like there is a lot of repetition in the beginning of Section VII

- ☐ Have to mention AES-CTR in the discussion as it is currently only referred to in the conclusion.

## I. INTRODUCTION

Today, when you want to use a data lake, you have two options. *Option 1*: Select a “fully managed all-in-one cloud platform” that integrates with data lakes and provides fine-grained access control. *Option 2*: Provide the external query engines the ability to interact with the data lake, but lose the ability for fine-grained access control. So either you choose *Option 2* and limit who the data can be shared with, based on whether they should only be able to access part of the data, or choose *Option 1* and limit the query engines the data scientists want to use.

The Object Stores, such as Amazon S3, being the data lakes, do not understand a granularity finer than file level. This limitation prevents external access control services from governing access to a finer granularity, such as column-based. If, instead, the mechanism for access control was moved within the data lake itself, this could enable fine-grained access control. In the research project “Reviewing options for fine-grained Role-Based Access Control in Data Lakes”, Trøstrup and Lucas[1], the solution called “Object Store Wrapper Service” (OSWS) was proposed.

The idea of OSWS is to encrypt the Parquet files stored inside the data lake using Parquet Modular Encryption (PME), which supports encrypting each column. The keys will be stored in a key vault and in wrapped format within the Parquet files, and will be mapped to roles, to limit which roles have access to specific columns. OSWS will expose an S3-compatible API, which can then decrypt the data for the “fully managed all-in-one cloud platforms” and the external query engines when getting normal S3 requests. Thus, regardless of

where to read the data from, it will have been ensured that they can only see what they are supposed to.

In this thesis, a PoC of OSWS will be created to prove that a system supporting third-party query engines, such as DuckDB, without modifications, can use OSWS, though, as quickly realized, interoperability with the “fully managed all-in-one cloud platforms” is partly a job that the respective companies have to allow. After developing the PoC and coupling a query engine to it, experiments will be conducted to evaluate the performance and **security of the system**, compared to available solutions. Metrics include end-to-end read and write latency, as well as the cryptographic overhead, in relation to the size of the Parquet file, introduced by the OSWS. Experiments will use data sizes representative of realistic data sizes.

#### A. Research Questions

Is it possible to create a wrapper service that ensures access control on column-level using RBAC and has interoperability with existing query engines and “fully managed all-in-one cloud platforms” without modifying them?

#### B. Objectives

This research paper will have the following objectives:

- 1) Create OSWS PoC using an underlying S3-compatible Object Store
- 2) Ensure OSWS enforces access control using RBAC outward using encryption
- 3) Make it compatible with “fully managed all-in-one cloud platforms”
- 4) Make it compatible with query engines
- 5) Benchmark OSWS to measure the latency it adds.
- 6) Reflect on design decisions and which alternatives would have been better

## II. RELATED WORK

#### A. Reviewing options for fine-grained Role-Based Access Control in Data Lakes

This thesis is a continuation of the research paper titled “Reviewing options for fine-grained Role-Based Access Control in Data Lakes”[1].

This was the foundational work to reflect on how OSWS could theoretically work. After implementing the OSWS, discoveries revealed that it did not work as initially planned, and as described in the research paper.

The biggest change from the research paper is that OSWS does not support third-party query engines decrypting locally, but has to get the decrypted Parquet through OSWS. As also mentioned later, “fully managed all-in-one cloud platforms”, such as Snowflake or Databricks Unity Catalog, are not guaranteed to be supported as they would have to whitelist a URL on which OSWS runs, and have thus not been able to be tested; **it is not certain that, given a whitelist, it would work.** Lastly, data lake-specific systems, such as DuckLake, are not supported either, due to it caching metadata and the Parquet file being modified in OSWS, discussed in more depth in Section VII. Though this is an issue that could be removed

in the future, it will require OSWS to have another system design than the one it currently has.

#### B. Membrane: A Cryptographic Access Control System for Data Lakes

Membrane, referenced in Trøstrup and Lucas[1], is a proposal to ensure FGAC at the cell level and encryption-at-rest in data lakes using a custom format, Kumar et al.[2], thus resembling what OSWS sets out to do. Besides making the original client responsible for handling the original encryption keys to provide access to others, it also forces the query engine to download the whole file to decrypt it with the provided view on its machine. The advantage of this is that once the query engine has the data, given the view, they only have to make computations on the relevant data. This differs from OSWS, where it actually runs computations on all data, but instead only gives the query engine the finalized data. The computation volume for Membrane is therefore correlated to the size of the view size, whereas for OSWS, it is related to the file size, but OSWS supports existing query engines which can interact with S3. Membrane also has finer-granularity access-control than OSWS, cell-level vs column level, but this comes at the cost of using a proprietary format.

#### C. One Stone, Three Birds: Finer-Grained Encryption with Apache Parquet @ Large Scale

Shang et al.[3] present a system for fine-grained encryption of Parquet files at Uber’s scale, solving encryption-at-rest and access control as a combined problem—similar to OSWS. Their approach uses column-specific encryption keys stored in KMS, with readers and writers performing cryptographic operations directly through a Parquet library extension.

The key difference is where the cryptographic operations occur. The Uber system requires extending the Parquet reader/writer libraries used by query engines, making it intrusive to adopt. OSWS performs all cryptographic operations in the Encryption Gateway, exposing a standard S3 API so that unmodified query engines can access data without library changes. The trade-off is that OSWS introduces a network hop and full-file processing overhead, while Uber’s approach avoids both but requires per-engine integration.

## III. BACKGROUND

Data lakes are not a new concept, but the term was first coined in 2011 by Dixon[4], and some of the terminology is a bit inconsistent. This section will describe the terminology used within this thesis to ensure consistency. A lot of the definitions will be taken directly from the previously written research paper, which will be referenced and quoted accordingly.

#### A. OSWS

Object Store Wrapper Service (OSWS) is the concept of a layer on top of an Object Store and which support S3-compatible API endpoints. It is meant to provide fine-grained role-based access control by encryption at the columnar level. This will be achieved by using Parquet Modular Encryption

(PME), and will only decrypt the columns which are authorized to the client requesting a given Parquet file.

### B. JIT Provisioning

Just-In-Time (JIT) Provisioning is an identity management process handling creating user accounts when they are needed. [5] This is needed during the initial user creation, see Section IV.C.g.

### C. RBAC

In Trøstrup and Lucas[1], Role-Based Access Control (RBAC) was defined as follows:

Role-based Access Control (RBAC) [6] is an approach to access control based around assigning permissions to roles, and grouping users into these roles. Access control is then managed by controlling what roles are allowed to do what, instead of managing each user individually. The “Core RBAC” model, as defined by Ferraiolo et. al.[7], consists of five basic elements: *users*, *roles*, *permissions*, *operations* and *objects*. A *user* is some actor, typically a human user, but could also be an autonomous actor, who needs to be authorized for some set of actions. A user is assigned one or more *roles*. A role is simply some named collection of authorizations; for example, a job title. A role is assigned one or more *permissions*, which are what give authorization to perform an *operation* on a secured *object*. [...] In addition, Core RBAC also describes *sessions*. A *session* is a mapping of one user to a subset of the roles they are authorized for; i.e. a user “activates” one or more roles that have the necessary permissions, when they need to perform a specific operation on an object. This allows for finer control of roles that are active in a given context, to prevent unnecessary access levels. Commonly, a hierarchy is introduced to allow for structuring roles which should inherit some base permissions; for example, a Doctor and Nurse role might inherit permissions from some base “Healthcare Worker” role. This is referred to as “Hierarchical RBAC” [7].

— Trøstrup and Lucas[1]

### D. OIDC

OpenID Connect (OIDC) is an identity authentication protocol based on the authorization OAuth 2.0 framework.[8] It provides developers with a standardized, simple way to verify the identity of users trying to access web applications. It allows users to authenticate using their existing accounts previously registered with an OpenID Provider, such as Microsoft Entra ID[9] or PocketID[10], which the web application can then contact to verify their identity. Thus, it eliminates the need to implement an authentication layer in the web application.

### E. KMS/KV

Key Management Service (KMS) and Key Vault (KV) are both services used to manage keys.[11], [12] KMS is widely used within data storage, and is what AWS use to refer to

their system, whereas Azure uses KV. This paper will use the term KV, as OSWS has been set up against Azure KV but could have used KMS, see more in Section IV.D.d. KV is a way to store cryptographic keys. When the cryptographic keys are within the KV, they can no longer be retrieved, and thus if data has to be encrypted/decrypted, it either has to be sent to the KV or envelope encryption can be used, so the KV is responsible for unwrapping the DEK, see more in Section III.H.

### F. Data Lake

In the previously written research paper, data lakes were defined as follows:

A “data lake” is a centralized repository of data, which could be structured, semi-structured or unstructured.[4] It differs from a traditional database in that a data lake separates compute and storage layers. The data itself is stored in object stores like Amazon S3[13], in file formats like CSV, Parquet<sup>1</sup>, etc. The compute nodes are simply any query engine, such as Apache Spark [14], that connects to the object store and uses the data. This could be a data scientist working on their own machine, who connects to a data lake and runs queries; or it could be a part of a managed all-in-one platform like Snowflake, see more in Section III.K. As the object store just contains raw data, both structured and unstructured, this can make it more complicated to query, especially for large data lakes. One approach to querying is “schema on read”, in which the query engine infers a schema at query time based on the files in the data lake. This is in contrast to “schema-on-write” used in traditional relational databases, where the schema is strictly enforced when writing. This doesn’t work well for data lakes due to the nature of storing raw data in object stores. Typically, data lakes introduce a *catalogue* on top of the object store that manages file schemas, metadata and snapshots, allowing for structured reads and more performant querying through techniques like partitioning, i.e. filtering unnecessary data. This has also been referred to as a *data lakehouse* .[15]

— Trøstrup and Lucas[1]

### G. Parquet

In the previously written research paper, Parquet files were defined as follows:

Parquet [16] is a columnar file format supported by many data processing systems.

A Parquet file is structured using row groups, which contain columns, along with a footer which contains metadata. The columns are structured as column chunks, which contain several *data pages*. A data page contains a header, which describes information like the size of the page, and following that, the actual values as bytes. The

<sup>1</sup>Section III.G defines Parquet

file footer contains metadata that specifies what is stored in the file and where, that is, where a reader should look to find a specific column, or the statistics of a column (e.g. min/max). Appendix 1.a shows a visual representation of a Parquet file that also uses encryption, which will be described later. The left side of the figure shows row groups, columns and data pages, along with their headers. The right side shows the footer of the file, with the file metadata containing metadata on each row group, which in turn contains metadata on the column chunks in that row group. The arrows link information about where to read, such as offsets, to where they point the reader. The objects that hold metadata, like statistics, and structural information, including physical offsets, such as the footer and page headers, are serialized using Apache Thrift[17], a language-agnostic serialization framework, and referred to as “Thrift Structs”. Essentially, the object that contains the information, for example, a `PageHeader` object (on the left side of Appendix 1.a) or a `ColumnMeta Data` object (on the right side of Appendix 1.a), is serialized into bytes using Thrift, which can then later be restored to the same object.[18]) Then, to read a Parquet file, the reader de-serializes the Thrift structures to get the necessary metadata, and then reads the actual data values using the information from the metadata.

— Trøstrup and Lucas[1]

#### H. Envelope Encryption (KEK, DEK)

KV does not support retrieving keys, and as a result, OSWS uses envelope encryption. This means that during encryption of a column, cryptographic keys for each are generated, called a data-encryption-key (DEK), which is used to encrypt the columns. Then, for the single Parquet file, a single key-encryption-key is generated, resulting in less data compared to if each DEK had its own separate KEK within KV. This KEK then encrypts all the DEKs, also called wrapped DEKs, and gets stored in KV. The wrapped DEKs are then stored in the metadata for the Parquet file together with a KEK id. When the columns are to be decrypted, the wrapped DEKs are sent to KV to be unwrapped, and the columns can be decrypted.[19]

#### I. TTL

Time-to-live (TTL) is a way to define the lifetime of some item within a dataset. When either the item’s TTL expires or the dataset is full, the item closest to its TTL is removed. [20] In OSWS, TTL defines the lifetime of unwrapped DEKs in memory. When the TTL is reached, the data will be removed and has to be re-fetched from the KV when needed again. OSWS defines two different TTLs as a way to follow general security guidelines, which can be read about in “NIST Special Publication 800-57 Revision 5 Recommendation for Key Management”[21], where OSWS uses a TTL of five minutes for DEKs authorized to admins, and the rest have a TTL of 15 minutes.

#### J. PME

Parquet Modular Encryption (PME) is a way to allow granular control of how the Parquet file data and metadata should be encrypted. Singular columns can be encrypted, and the footer as well. For OSWS, footer encryption is not enabled, as it was outside of the scope. When encryption is used, serialized Thrift structs are encrypted using the given cryptographic key, and the data pages themselves are encrypted as well. In OSWS, it is saved as a tuple in the footer, containing a reference to the KEK in KV and the wrapped DEK. This allows OSWS to get the DEK unwrapped to then decrypt the column.[18]

#### K. Fully managed all-in-one cloud platforms

In this paper, “Fully managed all-in-one cloud platforms” will be used to define solutions such as Snowflake and Databricks. Access control within those solutions is managed by the query engines together with the catalogue. This means the query engine is trusted to filter out rows that are not permitted, and it then masks or removes those values, which is possible as the platforms themselves manage it.

#### L. Vended Credentials

“Vended credentials” is a way to manage temporary access to files within S3, but the issue is that it can only gate access on the file level, based on limitations in S3. This is the way Lakekeeper and Apache Polaris currently manage their access grants.[22], [23]

## IV. SYSTEM DESIGN

What the current OSWS solves is that query engines, which read metadata from Object Store and do not use cached values, will have columnar access control enforced on the data they read. This ensures that, from the query engine’s perspective, they are using S3 directly, but in reality use OSWS, and it can enforce whatever RBAC rules it wants on the clients. So if you have a dataset with columns `name`, `birthday`, and `relational status`, and the query engine is not granted access to `birthday`, it will receive, all columns, but `birthday` will have null or another default value.

Below, the design choices will be described in more depth, together with deeper technical information.

#### A. Architecture Overview

OSWS is set up as a wrapper for S3 and thereby provides S3-compatible API endpoints for clients. The purpose of OSWS is to provide column-level access controls to S3, whilst not modifying the S3 API. This ensures that most S3-compatible query engines can use OSWS without modification and will only read what they are supposed to.

The system is built on ASP.NET Core 10 using the minimal API pattern[24], which replaces traditional MVC with lightweight endpoint definitions. OSWS uses: PME via. Parquet Sharp, PostgreSQL for RBAC metadata, Azure KV for key management, and a configurable S3-compatible Object Store (tested with Cloudflare R2 and Digital Ocean Spaces for

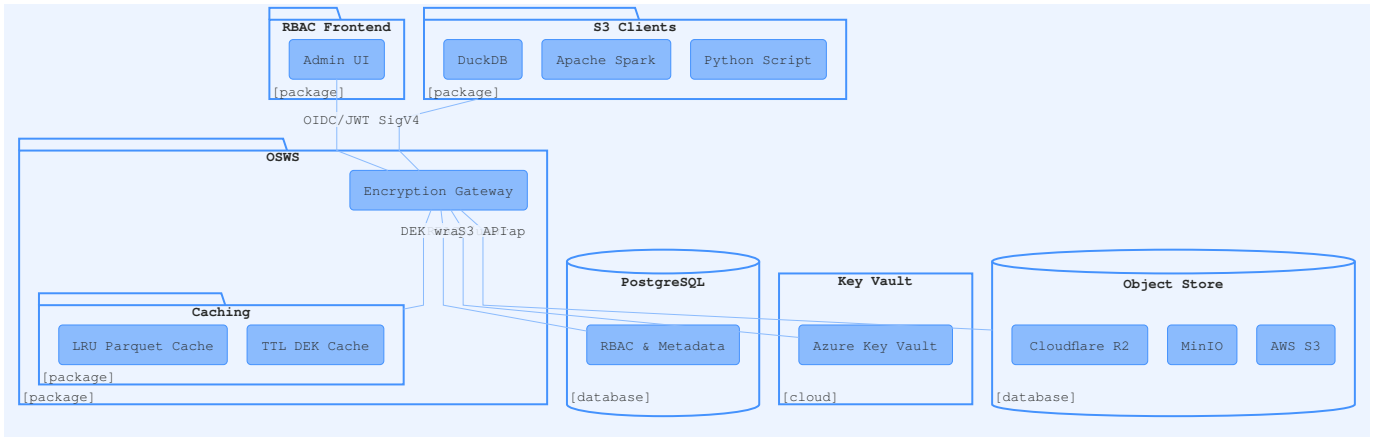


Figure 1. Architecture of OSWS showing components, external services, and communication paths

benchmarking).<sup>2</sup>

TABLE I  
OSWS SYSTEM COMPONENTS AND THEIR RESPONSIBILITIES

| Component          | Responsibility                                                                                                                                                               |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Encryption Gateway | S3-compatible API host. Handles incoming GetObject/PutObject requests, enforces AWS Signature V4 authentication, and handles encryption, decryption, and column filtering.   |
| Parquet Solver     | Encrypts and decrypts Parquet files using PME with ParquetSharp. Generates DEKs, wraps them via the Key Manager, and masks unauthorized columns.                             |
| Key Manager        | Manages cryptographic keys through Azure KV (or an internal provider). Wraps/unwraps DEKs using RSA-2048 KEKs stored in the vault.                                           |
| RBAC Store         | PostgreSQL database storing users, roles, role assignments, role inheritance, column permissions, and S3 credentials. Evaluated per-request to determine authorized columns. |
| Admin Frontend     | React web UI for managing roles, permissions, and users. Uses OIDC authentication and provides a SQL-like query editor for RBAC operations.                                  |

OSWS consists of the five main components, see Figure 1 or Table I.

The **admin frontend** is for RBAC, thus managing roles, permissions, and users within the OSWS system, more specifically within the **RBAC store**, which is the database for storing these relations. Here, it can be defined what columns a user can access later on and, by extension, which columns will be masked.

The **encryption gateway** is the entry point for retrieving and putting files into the actual S3. It is responsible for delegating work to the other components, so encrypting and decrypting files, partially, is possible. Whenever it receives a GET request, it will first get the file from S3 and see which columns the client is able to read, by using the **RBAC store**. Then it will use the **Parquet solver**, which in turn will handle trying to identify which DEKs are in the Parquet file. It can

then go through the **key manager** to get the unwrapped DEKs based on permission level; it will check locally first, and if not stored, go to KV to get them unwrapped there, and store them for later use. After the key manager returns the unwrapped DEKs, the Parquet solver can decrypt the columns and mask out the ones it cannot decrypt. Now the encryption gateway can return the Parquet file, showing only the information that the client has access to.

When the encryption gateway gets a PUT request, it first gets the user's base role. This is mistakenly done by taking their first role, but should have been tied to their base role instead. Then the Parquet solver is given the file and role, so it can generate new DEKs per column and a KEK for the Parquet file. The RBAC store is updated with the new columns and key references, and the file is encrypted afterwards. It also handles wrapping the keys initially so the wrapped DEKs can be put into the metadata, and KEK is given to the key manager to insert into KV.

This allows OSWS to ensure fine-grained role-based access control in data lakes, but of course, there are more in-depth design choices that have been made to make this work, both for better and worse.

## B. Design Choices

### a) Database Design:

An Entity Relationship describing the database design can be seen in Figure 2. The database is mostly used for RBAC metadata; however, it also stores credentials used for AWS Signature V4 request signing, see Section IV.C.f for more info on SigV4, and external identity information from the OpenID Provider(s).

### b) Role Hierarchy:

OSWS also supports Hierarchical RBAC through the `RoleInheritance` tables. This is a self-referential table in which a hierarchical relation is created through the `ParentRoleId` and `ChildRoleId` foreign keys, so that a *parent* inherits from a *child*, see `RoleInheritance` in Figure 2. This means that all permissions of *child* also become permissions of *parent*. To retrieve the full set of effective roles for a user, a recursive SQL query traverses the hierarchy. This can be seen in Listing 1.

<sup>2</sup>Codebase available at [github.com/lucasfth/osws](https://github.com/lucasfth/osws)

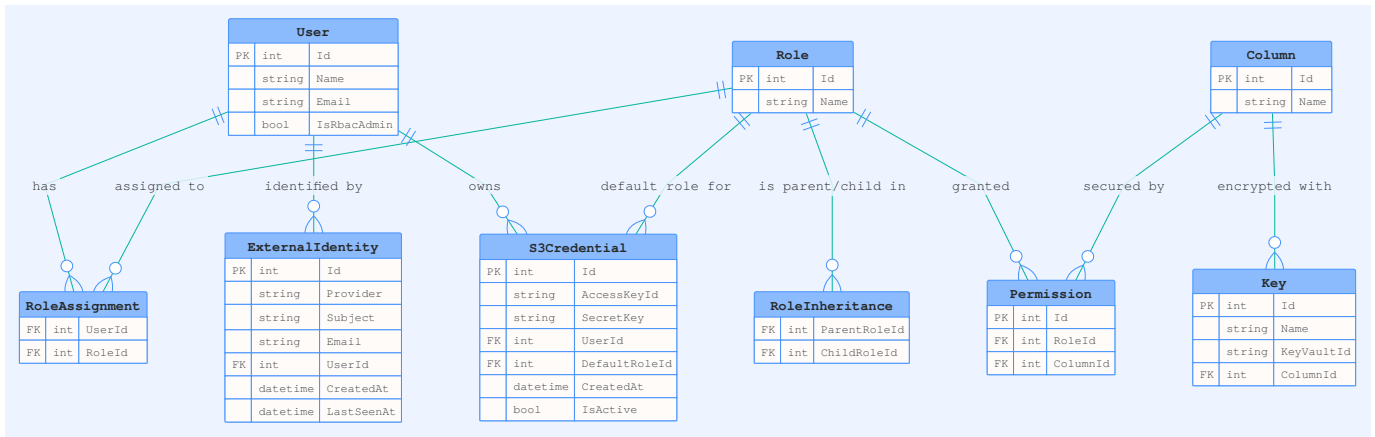


Figure 2. Entity Relationship Diagram showing the design of the database

```
WITH RECURSIVE effective AS (
  SELECT ra."RoleId" AS "Id"
  FROM "RoleAssignments" ra
  WHERE ra."UserId" = {userId}
  UNION
  SELECT ri."ChildRoleId"
  FROM "RoleInheritances" ri
  JOIN effective e
  ON ri."ParentRoleId" = e."Id"
)
SELECT DISTINCT r."Id", r."Name"
FROM "Roles" r
JOIN effective e ON r."Id" = e."Id"
```

Listing 1. A recursive SQL query to get the full set of effective roles given a userId.

#### c) Column-Level Permissions:

Each *Permission* record maps a *Role* to a *Column*. This means that a column is decrypted if and only if one of its roles grants access to the given column. This ensures fine-grained access control. If a Parquet file *P1* with columns *name*, *age*, and *civil\_personal\_register* exists. Then a user *U1* and a user *U2* can respectively give access to role *R1* and *R2*. *R1* provides access to only *name* and *R2* only to *age* and *civil\_personal\_register*. Then *U1* would see a table full of valid *names* whilst *age* and *civil\_personal\_register* are null/default values, while *U2* would see *age* and *civil\_personal\_register* values.

### C. Caching

There are two tiers to the caching system used within OSWS with the intention of minimizing latency from remote services.

#### a) Encrypted File Cache:

Given the configuration of OSWS, the Parquet files fetched from the object store can be cached. It uses the LRU (Least Recently Used) policy to cache the encrypted Parquet files on the local file system. They are keyed with SHA256(bucket::key), see OSWS.ParquetSolver/Helpers/EncryptedFileCache.cs.

#### b) DEK Cache:

Unwrapped DEKs are cached in-memory to avoid repeated KV calls, as it is one of the major latency attributes, discussed

later. The cached DEK is keyed with the KEK identifier, and in the configuration, the TTL can be defined for regular and admin users, as admin users' keys are most often more privileged. The cache enforces a maximum capacity and evicts the expired entries first, and then the oldest entries by expiration date.

#### c) Envelope Encryption:

To encrypt the Parquet columns, OSWS needs to use DEKs and KEKs to achieve envelope encryption[25]. This deviates from the original proposed solution in Trøstrup and Hanson [1], as KV does not support key retrieval, and the overhead of sending a Parquet column to KV each time for decryption and encryption is high. By using envelope encryption, OSWS can cache the unwrapped DEKs with a specified TTL to allow quicker decryption. This results in a solution with little overhead and ensures column-level access control.

#### d) Parquet Solver:

Parquet Solver handles the cryptographic operations for the Parquet files. Important design decisions made here are that it uses Parquet Sharp. Parquet Sharp was chosen as it supports PME, but does not support partial decryption/reads of Parquet files. This results in the fact that when a Parquet file has to be decrypted, it essentially decrypts and copies over all the columns into a new Parquet file. But it does not support copying over the columns which are not supposed to be decrypted. As a result, dummy columns are created, and given the setup of OSWS, they are encrypted and copied over into the new Parquet file.

This is one of the design choices made that later created more problems than it solved.

#### e) Authentication:

Due to S3 requiring signage, the calls from the clients going to OSWS are signed as well, and as a result, OSWS cannot reuse the signature and encrypt the Parquet columns. Internal authentications were therefore needed, and OSWS then has its own signature for S3.

#### f) AWS Signature V4 (S3 API):

Because OSWS had to use an S3-compatible API, it also had to use the same validation as S3. A custom SigV4AuthenticationHandler has been created to parse the authorization header and extract the AccessKeyId. The keys corresponding to the



S3Credential record are then looked up in the internal PostgreSQL, see the S3Credential table in Figure 2. The signature is verified using the stored secret key. On success, a ClaimsPrincipal is created carrying the user's database ID, name, email, and default role.

This follows the AWS specification "Authenticating Requests (AWS Signature Version 4)"[26].

#### g) OIDC:

The React frontend, for handling RBAC, uses OIDC for the authentication flow. The OIDC provider chosen for OSWS is Pocket ID, as it is simple to set up, is open-source, and uses passkeys, which are more secure than standard MFA, as "common multifactor authentication methods can be intercepted or relayed"[27].

The backend still supports other OIDC providers. To use other providers, OSWS.WebApi/appsettings.json needs to be updated to reflect the change, and the frontend .env has to point to the specified authority and client ID.

On the first login, the api/me endpoint triggers JIT provisioning, and a new User record and ExternalIdentity record are created in the database. Subsequent logins will synchronize the OIDC provider's claims.

#### D. RBAC

In Section III.C, RBAC is defined to consist of: *users*, *roles*, *permissions*, *operations*, *objects* and *sessions*. OSWS implements RBAC mostly following this, but it differs in a few ways. In Figure 2, the core RBAC entities are shown: the tables User, Role, RoleAssignment, RoleInheritance, Column, and Permission. User and Role have a many-to-many relationship through RoleAssignment. Likewise, Role and Column have a many-to-many relationship through Permissions. The Column table can be seen as the *object* of Core RBAC, as it is currently the only secured object to which access is controlled. There is, however, no concept of *operations*, only whether or not access is granted. There is also no formal concept of *sessions* as described by Ferraiolo et al.[6]; however, when a user is authenticated through their S3 Credential, their assigned roles are loaded. In this way, it can be seen as an activation of all the users assigned roles.

##### a) Key Hierarchy:

The KEK sizes are specified within the KV that the admin chooses. Due to having student credits available in Azure, which was only available for a low-cost KV, RSA-2048 was used. The KEKs are created in OSWS (inside OSWS.KeyManager/Providers/AzureKeyVaultProvider), but after that, the KEK never leaves KV again, and are then called by OSWS to wrap and unwrap the DEKs.

For the DEKs OSWS, create these themselves. These symmetric AES keys have sizes 128, 192, or 256, and are created during the encryption of Parquet files.

##### b) Administrative Endpoints:

Endpoints for admin endpoints are restricted to only users who have the IsRbacAdmin flag, seen in the User table in Figure 2. This field is populated from the

ClaimsPrincipal when provisioning the user in the /api/me route. Thus, it requires the OIDC Provider to include this claim on users who should have admin access. Alternatively, it can be set manually.

TABLE II  
ADMIN ROLE MANAGEMENT ENDPOINTS PROVIDED BY OSWS

| Method | Path                                          | Description                                                     |
|--------|-----------------------------------------------|-----------------------------------------------------------------|
| GET    | /api/admin/columns                            | Get all columns                                                 |
| POST   | /api/admin/columns/{columnId}/roles/{roleId}  | Grant access on column with ID columnId to role with id roleId  |
| DELETE | /api/admin/columns/{columnId}/roles/{roleId}  | Revoke access on column with ID columnId to role with ID roleId |
| GET    | /api/admin/roles                              | Get all roles                                                   |
| POST   | /api/admin/roles                              | Create a role                                                   |
| DELETE | /api/admin/roles/{id}                         | Delete role with ID id                                          |
| POST   | /api/admin/roles/{parentId}/inherit/{childId} | Make role with ID parentId inherit role with ID childId         |
| DELETE | /api/admin/roles/{parentId}/inherit/{childId} | Delete inheritance of role parentId from role childId           |
| GET    | /api/admin/users                              | Get all users                                                   |
| POST   | /api/admin/users/{userId}/roles/{roleId}      | Grant role with ID roleId to user with ID userId                |
| DELETE | /api/admin/users/{userId}/roles/{roleId}      | Revoke role with ID roleId from user with ID userId             |

##### c) Frontend Architecture:

A frontend for interacting with the endpoints described in Table IV and Table II was built using Typescript-React. To quickly create a user-friendly working prototype, UI components from the open-source project shadcn [28] were used. A user can log in using Pocket ID, as described in Section IV.C.g. Here, the user can create credentials to be used for the S3-Compatible API endpoint. If the user is an RBAC Admin, they get access to the admin panel for managing roles. The admin panel includes a table view for viewing currently existing users, roles, columns and permissions. To interact with the endpoints described in Table II, a "query editor" was built to manage RBAC operations in an SQL-like syntax, reminiscent of the syntax used to manage permissions in e.g. PostgreSQL.<sup>3</sup> Using the JavaScript library peggyjs [29], a simple grammar was written to parse statements into API calls. Some example statements to configure RBAC permissions and the API calls they parse can be seen in Listing 2.

<sup>3</sup><https://www.postgresql.org/docs/current/sql-grant.html>

```

CREATE ROLE admin;
=> POST /api/admin/roles { name:
"admin" }
CREATE ROLE intern;
=> POST /api/admin/roles { name:
"intern" }
GRANT intern TO ROLE admin;
=> POST /api/admin/roles/1/inherit/2
GRANT ACCESS ON name TO intern;
=> POST /api/admin/columns/1/roles/1
GRANT ACCESS ON ssn TO admin;
=> POST /api/admin/columns/2/roles/2
GRANT admin TO USER alice;
=> POST /api/admin/users/1/roles/1

```

Listing 2. Example statements and their parsed API calls

OSWS consists of the five main components, see Figure 1. The data flow for a read request is as follows: the client sends an S3 `GetObject` request to the Encryption Gateway, which authenticates via SigV4. Then the gateway retrieves the encrypted Parquet file from the Object Store - checking the file cache first. The DEKs, which the client has access to, are sent to KV to get unwrapped if they were not found in the DEK cache. The “Parquet Solver” then decrypts the available columns and replaces any that cannot be decrypted. The new Parquet file is now returned to the client.

For a write request, the client uploads a Parquet file via `PutObject`. The “Parquet Solver” encrypts the columns with newly generated DEKs, wraps them via KV, and stored the wrapped DEKs inside the Parquet files. Columns and permissions are persisted inside the RBAC database.

#### d) Key Manager:

The Key Manager provides an abstraction over cryptographic key storage and the relational model.

- **AzureKeyVaultProvider:** Implements `IKeyVaultProvider`. It creates keys (which are defined in `OSWS.Common/Configuration/EncryptionSettings`) in Azure Key Vault, performs the wrap and unwrap of the DEKs server-side, using the algorithm defined within `OSWS.ParquetSolver/Helpers/Cryptography`.
- **InternalKeyVaultProvider:** Implements `IKeyVaultProvider`. It works the same as `AzureKeyVaultProvider` but runs on OSWS itself.

As long as `IKeyVaultProvider` is implemented together with a key vault provider, which means it supports the following functions: encrypt, decrypt, get key info, and list keys, most providers should be able to be used.

- **OswsContext:** Implements `DbContext`. Defines the relational schema over PostgreSQL, including users, roles, role assignments, role inheritance, permissions, columns, keys, external identities, and S3 credentials.

#### e) Shared Libraries:

- **OSWS.Models** defines all the DTOs and entities used within the solution.

- **OSWS.Common** contain classes containing settings for the project, including `EncryptionSettings`, `CacheSettings`, `S3Settings`, `KeyVaultSettings`, and `RateLimitSettings`. They are bound to `appsettings.json` at startup, and they also include validation logic for the configuration.
- **OSWS.Library** has utility helpers for AWS credential normalization, S3 metadata translation, HTTP range request parsing, parameter validation, and XML extensions.

#### f) Encryption Flow:

The encryption flow works as follows:

- 1) Client uploads unencrypted Parquet file, via `PUT /s3/{bucket}/{key}`.
- 2) OSWS creates an RSA-2048 key and is tagged with the uploading user’s role.
- 3) For each column designated for encryption, an AES DEK of specified sizes is generated and encrypted, within `OSWS.ParquetSolver/Helpers/Cryptography`.
- 4) Each DEK are then wrapped by using the KV, then serialized, and put into the footer of the Parquet file.
- 5) The encrypted Parquet file is then written using Parquet Sharp.
- 6) Parquet file is then sent to the S3-compatible object store.
- 7) Columns, key IDs, and permissions are persisted in local PostgreSQL.

#### g) Decryption Flow:

The decryption flow works as follows:

- 1) The client requests a Parquet file, via `GET /s3/{bucket}/{key}`.
- 2) OSWS fetches the encrypted Parquet file, first tries in local cache, then if not found, it goes to S3-compatible object store (see `OSWS.WebApi/Services/Services/S3ObjectFetcher`)
- 3) Wrapped DEKs are read from the Parquet footer.
- 4) For each wrapped DEK, OSWS checks the in-memory DEK cache. On a cache miss, it calls the KV decrypt method to unwrap the DEK and caches the result.
- 5) The user’s effective roles are computed via Listing 1.
- 6) The permitted columns are now decrypted, while the others are replaced by dummy columns<sup>4</sup>
- 7) The decrypted Parquet stream is returned to the client.

## E. API

OSWS exposes three different endpoint groups.

#### a) S3-Compatible Endpoints:

A subset of S3 required endpoints is implemented to allow for the object store operations. These operations can be seen in Table III.

<sup>4</sup>Due to limitations in Parquet Sharp, it is not possible to leave the encrypted column alone, and thus has to be replaced.



TABLE III  
S3-COMPATIBLE ENDPOINTS PROVIDED BY OSWS

| Method | Path                | Description                                                           |
|--------|---------------------|-----------------------------------------------------------------------|
| GET    | /                   | List buckets                                                          |
| GET    | / {bucket}          | List objects in a bucket                                              |
| PUT    | / {bucket}          | Create bucket                                                         |
| DELETE | / {bucket}          | Delete bucket                                                         |
| POST   | / {bucket} ?delete  | Delete multiple objects in bucket                                     |
| HEAD   | / {bucket}          | Retrieve bucket metadata                                              |
| HEAD   | / {bucket} / {key}  | Retrieve object metadata                                              |
| GET    | / {bucket} / {*key} | Retrieve an object (with decryption and column filtering for Parquet) |
| PUT    | / {bucket} / {*key} | Store an object (with encryption for Parquet)                         |
| DELETE | / {bucket} / {*key} | Delete an object                                                      |

Currently, non-Parquet files pass through the encryption step, and the endpoints defined are only to allow operations made by most query engines, and thus suffice for making an MVP. In the future, this should be extended to also allow non-Parquet files to be encrypted and then include the KEK reference within the Parquet file referencing it, and the endpoints should also be extended.

#### b) Application Endpoints:

OIDC-protected endpoints for the web frontend. See the endpoints in Table IV.

TABLE IV  
APPLICATION ENDPOINTS PROVIDED BY OSWS FOR WEB FRONTEND

| Method | Path                   | Description                                          |
|--------|------------------------|------------------------------------------------------|
| GET    | /api/me                | Current user profile (triggers JIT provisioning)     |
| GET    | /api/credentials       | List the user's S3 credentials                       |
| POST   | /api/credentials       | Create a new S3 credential (access key + secret key) |
| DELETE | /api/credentials/ {id} | Revoke a credential (soft delete)                    |

#### F. Limitations

OSWS set out to provide columnar access control to 3rd party query engines and “fully managed all-in-one cloud platforms” by encrypting columns individually using PME, provided through the Parquet Sharp library. But due to early design decisions, this was not possible. As OSWS uses Parquet Sharp and stores cryptographic keys within the metadata of the Parquet files, a few issues appear with some query engines. First, Parquet Sharp uses Apache Arrow, which itself supports writing metadata.[30] This in itself makes sense, as it is possible by looking at e.g. `created_by` to see who created the Parquet file.[31] But as Parquet Sharp needs to write when encrypting the Parquet file, the file is essentially changed with new metadata, and it will no longer be the same Parquet file that the query engine inserted. This results in the

fact that if the query engine stores metadata for ranges that are incorrect, which break their range requests. On top of that, OSWS saves cryptographic-related metadata within the footer, and that is yet another modification. As such, for this to work with all query engines, it would have to be ensured that all ranges of data remain the same, also after encryption.

The same issue is probably prevalent on most “fully managed all-in-one cloud platforms”, but it is not possible to test, due to it needing to have a URL whitelisted by the providers to test the OSWS system. But given documentation of e.g. Snowflake, it seems to point towards incompatibility.[32], [33]

In Section VII, this will be discussed in more depth and possible solutions to bridge these two design incompatibilities.

Lastly, initially, it was intended to make OSWS return the encrypted columns. But due to the limitations in Parquet Sharp, it was not possible, and the current solution is instead to create null/default (dummy) data entries for the column. There is an unimplemented method ready to encrypt this dummy data to simulate the idea of returning encrypted data, but it was not implemented due to being redundant since OSWS already had the previously mentioned limitations.

## V. METHODOLOGY

To ensure OSWS could live up to the intended use-cases and to check the viability of it, both benchmarking and e2e tests are quite important. This section will cover both the setup of the benchmark and the setup of the e2e test.

All source code is available at [github.com/lucasfth/osws](https://github.com/lucasfth/osws), and also describes how to get it running.

#### A. E2E Tests

As one of the goals of the OSWS was to enable third-party query engines to connect without any modification, a test confirming this behaviour is needed.

An e2e test will be created that asserts different query engines can connect and perform queries as normal, with OSWS providing column-level security based on the test user's roles. The e2e test suite will contain a seeding step that sets up the necessary users and roles. Python scripts will be used to orchestrate setting up roles and permissions, and asserting that the fetched data does not contain unauthorized data. **passer det ift titanic?** The test will use the “Titanic” sample dataset<sup>5</sup> and permissions will be configured so two roles have limited access, but to different columns. One role will inherit both and have direct access to all other columns, thus having full access. This will then test the role hierarchy feature. Then, the following will be used to fetch the Parquet file through different users and assert that only the permitted columns are viewable:

*Python script:* A Python script using an S3 client like `boto3` and a Parquet reader like `pyarrow` to fetch a file from OSWS.

*DuckDB:* A DuckDB instance will query the parquet file through OSWS.

*Apache Spark:* An Apache Spark instance (through PySpark) will query the parquet file through OSWS.

<sup>5</sup><https://www.agentsfordata.com/app?sample-id=titanic&tab=local>

TABLE V  
MAIN CONFIGURATIONS OF BENCHMARKS THAT ARE TO BE RUN

| Category | Benchmark                 | Configuration                                                       | Measurements |
|----------|---------------------------|---------------------------------------------------------------------|--------------|
| E2E      | S3 (Spaces)               | No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)            | Latency      |
| E2E      | OSWS +encryption +caching | No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)            | Latency      |
| E2E      | OSWS +encryption -caching | No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)            | Latency      |
| E2E      | OSWS -encryption -caching | No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)            | Latency      |
| Micro    | Permission Service        | No. flat hierarchy roles: 4, 64, 256                                | Latency      |
| Micro    | Permission Hierarchy      | No. hierarchical depth roles: 0, 4, 16, 64                          | Latency      |
| Micro    | Unwrapping                | Key size: 128,192, 256 bits                                         | Latency      |
| Micro    | Decryption                | No. rows: 1,000, 10,000, 250,000, 1,000,000, 2,000,000 (50 columns) | Latency      |

Each of the different tools should assert that only the permitted data is viewable.

### B. Benchmarking

All benchmarking of OSWS was run on a Digital Ocean VM with access to 8 cores and 16GB of DRAM, see Appendix 1.b, together with Digital Ocean’s own Object Store (Spaces), which is S3-compatible. This helps ensure minimal latency between the compute and storage. For benchmarks using a key vault, Azure KV was used. All services were kept in the same region to further ensure minimal network latency. The version of OSWS used was [V2026.0.0 Alpha](#).

The test parquet files used in the following benchmarks can be defined as follows:

- *Parquet Size Tiny*:~500KB Parquet file with 50 columns and 1,000 rows.
- *Parquet Size Small*:~5MB Parquet file with 50 columns and 10,000 rows.
- *Parquet Size Medium*:~125MB Parquet file with 50 columns and 250,000 rows.
- *Parquet Size Large*:~500MB Parquet file with 50 columns and 1,000,000 rows.
- *Parquet Size X-Large*:~1GB Parquet file with 50 columns and 2,000,000 rows.

The five Parquet files are auto-generated by a script that is also available in the source code. The configuration of the files was chosen to reflect a somewhat realistic distribution of different sizes of Parquet files. Various openly available datasets were compared to settle on the choice of 50 columns. For example, several real-world example datasets are available from [ClickHouse](#) contain much fewer than 50 columns [34]. Another is a dataset collecting 1.1 billion taxi and Uber rides in New York City, which contains 51 columns [35]. 50 columns seemed like a realistic number of columns for a large OLAP dataset, and was used for the sizes.

#### a) Micro Benchmarks:

The micro benchmarks that will be run will show how each major part of OSWS performs to get insights into how much they add of latency. The following micro-benchmarks will be run:

- 1) Permission Service
- 2) Permission Hierarchy
- 3) Unwrapping

#### 4) Decryption

*Permission Service* will measure the latency of how long it takes to get a user’s effective roles to the given columns. The columns stay at a constant 100, the hierarchy (meaning roles pointing to roles) stays at zero, the roles assigned to the user are varied from four, 64, and 256 roles, and they are each assigned, on average, access to 80% of each of the columns.

*Permission Hierarchy* will measure the latency of how long it takes to get a user’s effective roles, but instead, roles are now linked to other roles and lastly to columns. The number of roles assigned is constantly at one, while the number of roles assigned in depth is varied from 0, 4, 16, and 64 and again, on average, each role is assigned access to 80% of the columns.

*Unwrapping* will measure the e2e latency for reading a tiny Parquet file (the small Parquet files are also run but ignored, but can be seen in Appendix 1.c) with a cold cache. The varying key sizes of 128, 192, and 256 bits will primarily affect the symmetric AES-GCM decryption.

*Decryption* will measure the latency for reading and decrypting a Parquet file, and the keys for decrypting will have been warmed and thus stored in cache; thus, KV is not taken into account here. The Parquet sizes that will be used are the same as with e2e benchmarking, being the tiny, small, medium, large, and extra large.

All of the micro-benchmarks were run 15 times to obtain statistically meaningful results.

See Table V for an overview of the benchmarks and their configurations, which will be run.

#### b) E2E Benchmarks:

The micro benchmarks show the individual parts’ contribution to latency, but the overall performance of OSWS is more important to identify if, from a performance perspective, OSWS is usable within production environments.

As OSWS inherently implements different design choices compared to vanilla S3, no standardized benchmarking has been chosen, such as [Warp](#) which is otherwise used by multiple other Fortune 500 companies.[36], [37] The reasoning for this is that OSWS implements its own caching layer and uses a KV; as such, the benchmark would have to ensure that it tests both the throughput of N instances of OSWS running and that the configuration of caching and encryption is considered thoroughly.

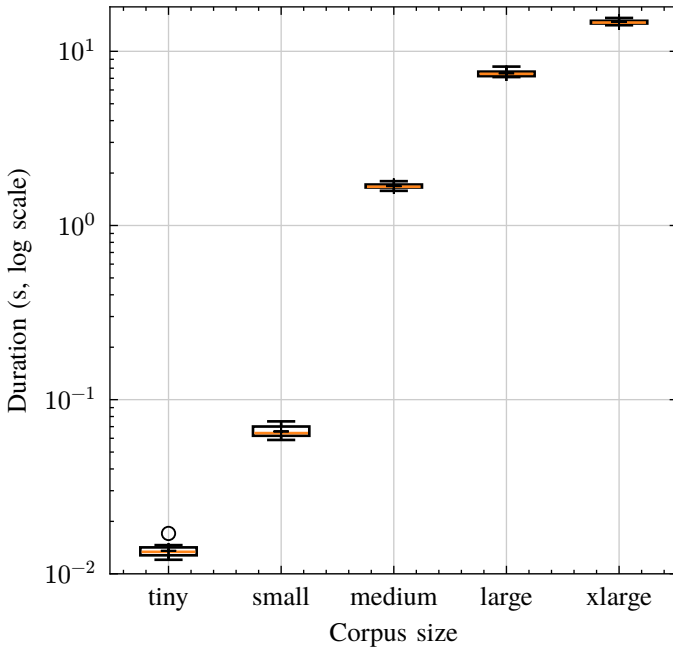


Fig. 3a. Decryption boxplot distribution per parameter (logarithmic  $y$  axis)

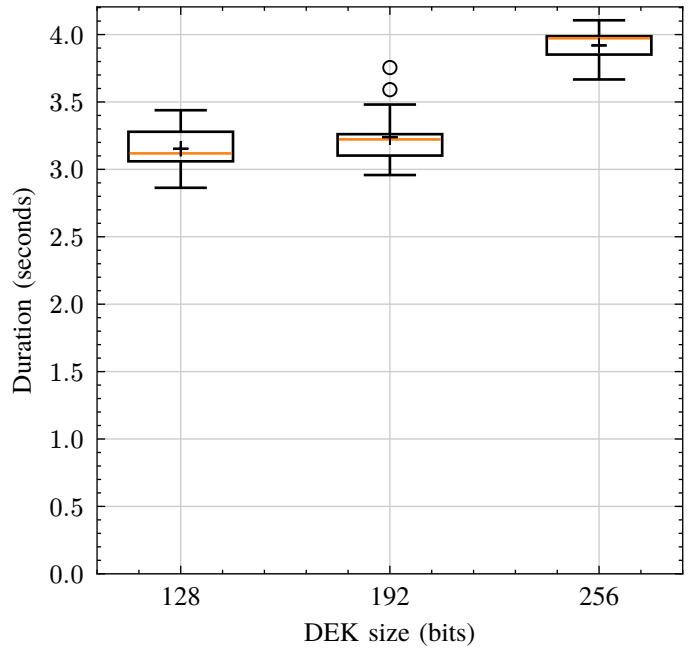


Fig. 3b. KeyUnwrap box-plot distribution per parameter set (tiny corpus only; linear  $y$  axis)

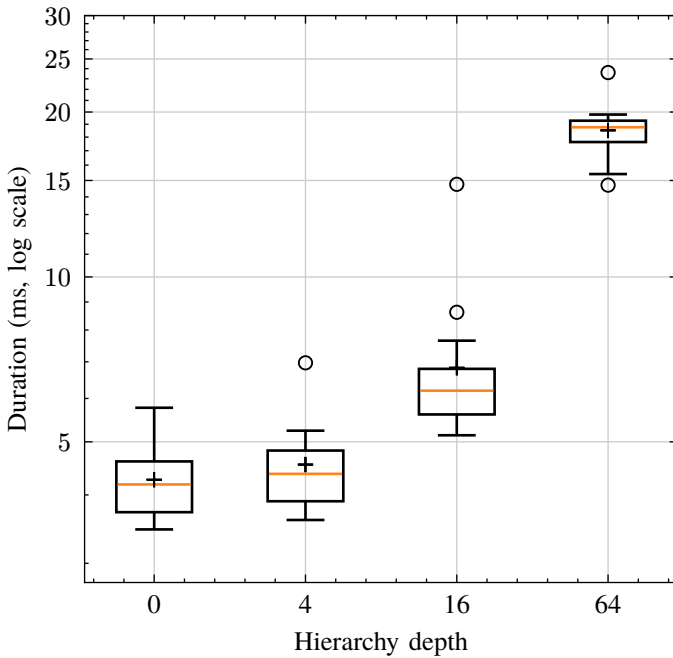


Fig. 3c. PermissionHierarchy box-plot distribution per parameter set (logarithmic  $y$  axis)

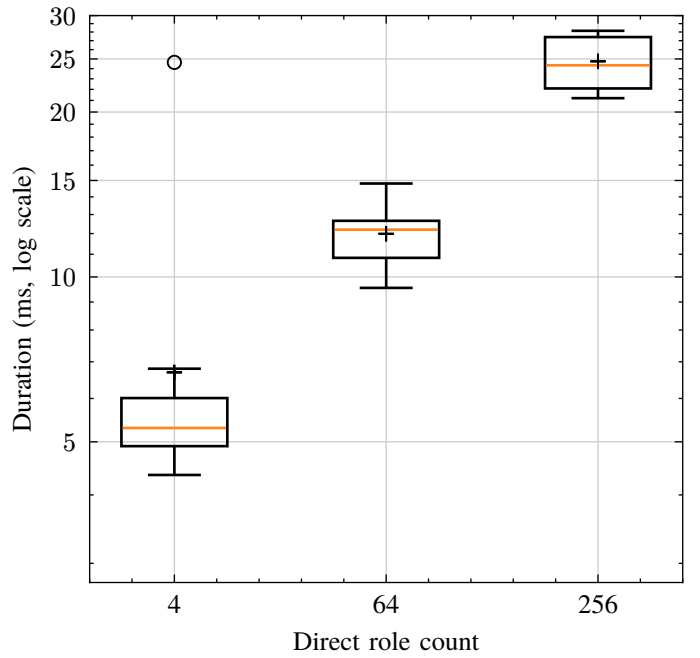


Fig. 3d. PermissionService box-plot distribution per parameter set (logarithmic  $y$  axis)

Figure 3. Box-plots for micro-benchmarks. Note that the  $y$  axes are not the same and some of the plots are logarithmic

OSWS also encrypts each column, resulting in the sizes of the Parquet files having a lot of significance for the end throughput, and Warp does not vary its Parquet file size. This results in inherently different throughputs and numbers, which are hard to compare OSWS against anything else.

Lastly, OSWS, as mentioned, does not support proper **range requests**, and it is then apparent that S3 will perform even better for ranged requests, where, regardless of the range or

full file request, S3 will perform close to the same.<sup>6</sup>

Python scripts will then run Parquet puts and gets against the following setups:

- Directly against an S3-compatible store
- Against OSWS with encryption and with caching
- Against OSWS with encryption and no file cache
- Against OSWS with encryption and no DEK cache
- Against OSWS with no encryption and no caching

<sup>6</sup>A solution to this is proposed within Section VII.

The metric for this will be latency, and for each file, the benchmark will be run for a total of 10 times, both cold and warm, allowing comparison of how slow or speed differences occur with and without caches.

## VI. RESULTS

This section will cover the results of running both the benchmarks and e2e tests described in Section V, and describe their significance.

### A. E2E Tests

An e2e test suite was written to test Python using PyArrow, DuckDB and PySpark against OSWS as an S3 endpoint. No modifications were made to the tools outside of changing the S3-endpoint to an instance of OSWS. The test suite shows that all three tools are successfully able to query a Parquet file and perform operations on the result. The users are identified through their credentials, and columns that the users' roles do not give access to are masked correctly.

This demonstrates that OSWS works as a drop-in replacement for S3 when working with query engines that fetch Parquet files directly and do not store metadata.

### B. Benchmarking

#### a) Micro-Benchmarks:

In Figure 3, all box-plots increase in duration whenever their role depth, flat roles, or the number of rows increases (Note that the  $y$  axis are not necessarily using the same scale and three of them are log-scaled). The latency for Permission Hierarchy Benchmark, see Figure 3c, and Permission Service Benchmark, see Figure 3d, both have a minor contribution to latency, which is expected for RBAC-related calls. These will not be referenced more as optimizations for these would not provide much benefit.

The major contributors to latency are decryption, see Figure 3a, and DEK unwrapping, see Figure 3b.

DEK unwrapping is close to constant. But the time is not something that can be improved much besides changing the KV service provider to a faster alternative, and then, as OSWS currently does, cache the unwrapped DEK heavily to use KV as little as possible. The values can be seen in more details in more details in Appendix 1.c, but essentially the  $p95$  time for unwrapping key size 256 of a tiny file is 4.11 seconds, with a mean of 3.92 seconds, meaning that caching the unwrapped DEKs is essential, as if not cached this time would have to be added to each time a column has to be decrypted.

Decryption is the other main contributor. With the tiny file, it has a  $p95$  of 17.1 milliseconds and a mean of 13.6ms, whilst for the medium and extra large, they are respectively,  $p95$  1.8 seconds, mean 1.68 seconds, and  $p95$  15.5 seconds, mean 14.7 seconds. So, essentially, to make OSWS usable for larger files, changes are needed in the decryption step. **Some of it might be possible to solve by using more asynchronous work, but also to make less operations.**

The large numbers for the decryptions are due to a few design decisions. First of V2026.0.0 Alpha relies on Parquet Sharp version 21.0.0, which, as mentioned, makes it neces-

sary, regardless of permission level, to copy over the whole Parquet file. To optimize this, a custom low-level reader would be needed, which could decrypt the columns in place. This would help, as the current decryption micro-benchmark also takes the copying into account, though the main improvement would be seen when columns are not authorized to the client, and can thus be left as is. So instead of the computations being directly related to the number of Parquet columns, they will be related to the number of authorized columns.

$$\text{Decryption: } O(|\text{columns}|) \geq O(|\text{columns}_{\text{authorized}}|) \quad (1)$$

#### b) E2E Benchmarks:

To begin with, comparison of cold GET latency with and without the DEK cache enabled is shown in Figure 5. Only tiny to medium is included due to the benchmark timing out on large and x-large Parquet files (>600s). **se om vi skal bruke large og xlarge selvom de timer ud jf martin feedback** However, even without large files, the DEK cache is invaluable even on small files, and on larger files, the system gets unusably slow without the DEK cache. At first, it seemed surprising that there was a difference between a cold GET and a warm GET. For the tiny GET, it was close to 120 times and 3562ms more slow on average, and for the medium, it was close to 4 times and 4148ms slower.<sup>7</sup> However, it makes sense given that the DEK cache is warmed up as the file is being read, since reading a row chunk will populate the DEK cache with the given columns of that chunk. Since the whole column is encrypted with the same DEK, once that column is reached again in a new row chunk, the cache is warm. This shows that the DEK cache is a must-have for OSWS. For that reason, the “no DEK cache” configuration is omitted from here on.

Figure 4 shows the results of the rest of the e2e benchmark suite. Originally, the intent was to have an x-large file size as well, but this turned out to be too slow to reasonably include in the plots. This also shows that there is a weakness when the files get to the ~1GB size.

However, looking at Figure 4a for a warm GET, it is again clear that the DEK cache is a must-have. On smaller files, all configurations of OSWS actually come close to the direct Digital Ocean fetch, though there is clearly a lot of overhead still. Interestingly, the “no encrypt” is seemingly a bit *slower* than the other configurations for the “tiny” and “small” configuration, even though it should just be the pure network overhead. However, disabling encryption also disables the file cache, which might be why it is slower – due to it having to go to Digital Ocean every time.

In Figure 4b cold GET is shown. Direct fetch is omitted here due to there being no “cold” path – this is the same as Figure 4a. Here, the “no encryption” configuration is an order of magnitude faster for the “tiny” and “small” configurations, which highlights the overhead of the decryption step. However, as the files get larger, the network overhead starts to show as well, when the “no encryption” configuration starts to approach the other configurations.

<sup>7</sup>Calculated from the results shown in Appendix 1.d

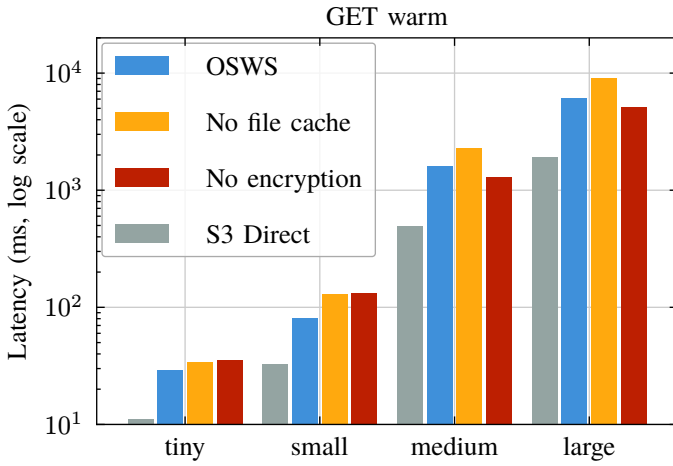


Fig. 4a. Median GET latency on warm cache

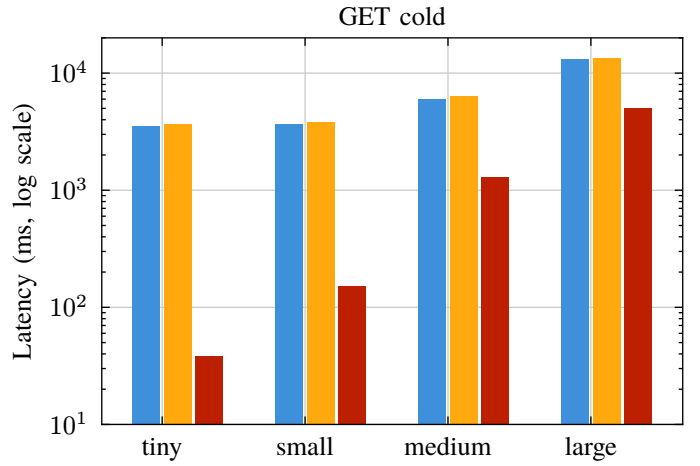


Fig. 4b. Median GET latency on cold cache

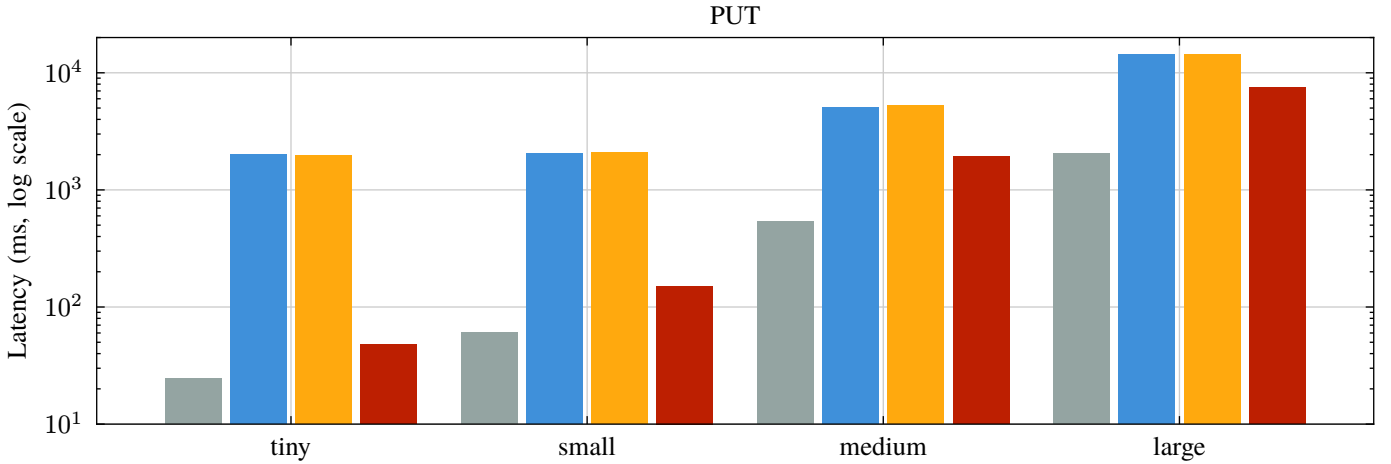


Fig. 4c. Median PUT latency

Figure 4. Plots showing median end-to-end latency measured during benchmarks. Grouped by file size. Note that legend is shared. (logarithmic  $y$  axis)

Generally, both Figure 4a and Figure 4b also show that the file cache is mostly useful when files are large; this is not surprising, as the encrypted file cache only helps with reducing S3-fetches, as the file must still be decrypted again. But when the network cost starts getting visible, the file cache matters.

Figure 4c shows the PUT latency. Unsurprisingly, there is massive overhead on the “tiny” configuration, as the direct upload is in millisecond-level, while OSWS must spend time encrypting the file and making calls to KV. Again, as files get larger, the relative overhead reduces as network transfer becomes a factor.

**Probably needs some specific numbers mentioned. Like takes this amount of sec for this size. And something like this setup is XXX sec/ms slower than S3 direct...**

## VII. DISCUSSION

OSWS set out to provide columnar access control to 3rd party query engines and “fully managed all-in-one cloud platforms” by encrypting columns individually using PME, provided through the Parquet Sharp library. Quick summary: During the encryption flow, Parquet Sharp modifies the metadata within, e.g. for the `created_by` value and metadata

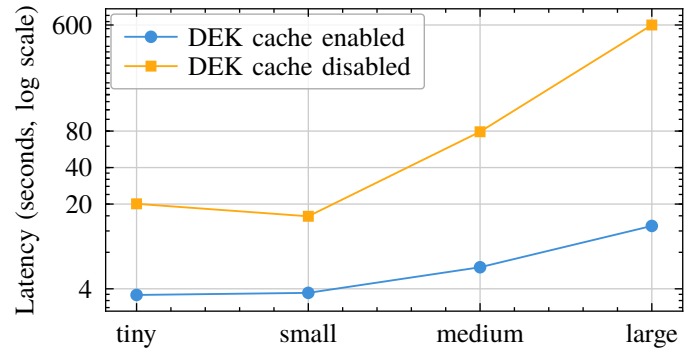


Figure 5. DEK cache impact on median cold GET latency. (logarithmic scale  $y$  axis)

is added for cryptographic operations, resulting in a changed Parquet file size. This eliminates query engines, which use the range modifiers based on their own cached ranges, as their ranged queries hit incorrect parts of the data within the Parquet file and do not take the modified Parquet file metadata into account, and probably also the “fully managed all-in-one cloud platforms”, but as mentioned in Section IV.F, have the same issue though not testable due to missing whitelisted URL. This then eliminates DuckLake[38] from using OSWS, as it caches the inserted ranges, and also Snowflake.



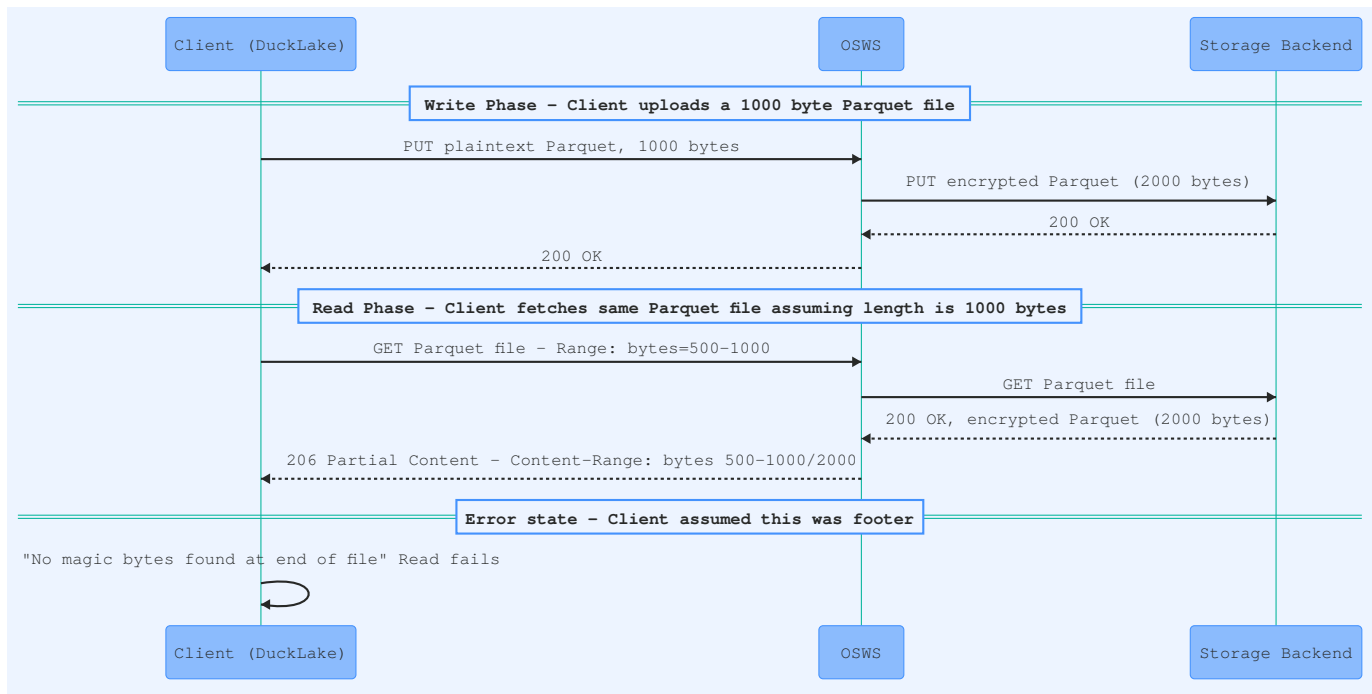


Figure 6. Sequence Diagram showing the DuckLake range request error case. File size and byte ranges are for illustrative purposes only.

Most of these incompatibility issues were from the initial implementation, and first became apparent on the final iteration of designing benchmarking and e2e tests, which, based on the research paper Trøstrup and Lucas[1], made sense, but after modifying it to work with the various providers, did not make sense anymore and had unnecessary overhead or created other problems, and as such this section will address those issues, define what should have been done instead, and define which performance improvements can be done in a future OSWS system.

#### A. Metadata

The original idea for OSWS was to allow 3rd party query engines to decrypt Parquet files themselves locally, while decrypting within OSWS for “fully managed all-in-one cloud platforms”. This would be achieved by storing key IDs within the footer of the Parquet files, and the query engines could then, through an endpoint in OSWS, request the given key and then decrypt the column locally, and OSWS would do the logic itself. As soon as it was identified that this was not possible with KV to retrieve the keys from it, OSWS was changed to use envelope encryption and store the wrapped DEKs, and the KEK reference in the footer, due to it being partly meant for that purpose. Se om vi skal opdatere med grunden her. Men ville jo argumentere at det allerede står i background. Men omvendt kan man sige det var vel også kun “umuligt” da vi regnede med ikke at bruge envelope enc. This means that when clients write files using the API, the Parquet file changes from the original file due to the newly added metadata. This does not affect *schema-on-read* måske skal dette begreb introduceres query engines like DuckDB and Spark; when they later fetch the file, they first perform a HEAD request to

get the size of the file, to know in what byte range to look for the footer. Thus, the reported file size is the *new* file size with the added metadata. The query engines are then able to perform range requests without any issues.

However, for *schema-on-write* tools like DuckLake that add a catalogue database, this poses an issue. When DuckLake writes a Parquet file, it records the size of the written file and the size of the footer in its catalogue database. This means it can later skip the HEAD request and fetch the footer directly. For OSWS, however, since the written Parquet file changes, this behaviour is broken. When DuckLake later attempts to fetch the footer of the tracked file, the byte ranges are no longer correct.

An illustration of this case can be seen in Figure 6. Here, it is shown what happens when DuckLake writes a file and later attempts to query it. The file sizes are chosen for illustrative purposes and are not real sizes. To begin, DuckLake PUTs a 1000-byte plaintext Parquet file, with a footer length of 500 bytes. These sizes are recorded in its catalogue database. When OSWS encrypts this file, it re-serializes the file to a new Parquet file with encrypted columns; each of these columns store their encrypted DEK in its metadata, adding file size – ending in a file size of 2000 bytes. This new, encrypted file is then stored in the storage backend.

When DuckLake later fetches this file, it looks in its own database and sees a file size of 1000 bytes and a footer length of 500 bytes. It issues a range request of bytes 500–1000, which it believes constitutes the footer of the file. This range request is propagated to the storage backend **Denne sektion skal omskrives til at være mere generisk; det nye metadata er ligegyldigt siden det ikke er med i filen der bliver sendt tilbage, problemet er fundamentalt at omskrive filen**

This breaks many query engines, where they themselves store size metadata, and range requests are not fully functional. The solution to this would most likely be to have an internal SQL server running in OSWS. This database should contain the following information per Parquet file inserted:

- 1) Internal File Identifier for footer<sup>8</sup>
- 2) Byte range of the start and end index
- 3) Internal File Identifier for Column Index<sup>8</sup>
- 4) Wrapped DEK
- 5) KEK reference
- 6) Row chunk size

Together with a full range of requests, support for more concurrency can be used. When OSWS get a request to read a file, it can start retrieving it. In the meantime, it can also go to the local database and find files which match 1. It can then see which keys the client has access to in the RBAC database, and then match the columns with 3, then for each send 4 to KV given 5.

Another improvement is when range requests are given, OSWS can figure out which ranges are related to which columns given 2 and then retrieve them from S3. Meanwhile, it can retrieve the unwrapped DEKs to be ready for decryption. Now for decryption, it can use the row chunk size 6 to be able to decrypt the small part, though only possible if AES CTR is used, more on this in Section VII.B.

So for metadata, it should only be saved within internal SQL, still, as the current solution relies on SQL, and then the encryption of the columns also needs to change.

## B. Encryption & Decryption

Currently, encryption and decryption are handled by Parquet Sharp, which is a .NET package which supports PME. Choosing this library was a mistake, as it handles encryption and decryption by reading everything (using cryptographic keys if provided) and copying it into a new Parquet file. If no cryptographic key is provided to an encrypted column, it fails. So here are two major issues with how it handles cryptography:

- 1) It reads and copies the entire file regardless of whether a single column has to be decrypted or all of them.
- 2) It does not allow for copying over encrypted columns.

Also, as Section VII.A, the size of the Parquet file changes upon encryption, due to PME, and thus creates issues for range requests.

All this could be solved by using the implementing a custom reader and writer for the Parquet files, which would use decrypt and encrypt in place, thus fixing 1, being able to leave non-authorized columns encrypted, thus solving 2, and not changing metadata and using the solution described in Section VII.A. Important for the encryption is that AES CTR, which does not use any padding, and thus is also a contributor to not modifying any part of the size of the file. Dworkin\_2001 The issues and things to account for when using CTR have been discussed by [40] and include the following:

- No integrity: CTR provides no message integrity, but can be handled by MAC.
- Error propagation: Bit flips are localized and not propagated. Though this issue should be solved in another layer.
- Stateful encryption: Important that the system does not reuse keys, but that is already how OSWS operates.
- Sensitivity to usage errors: Counter-values are not to be reused, but given that OSWS generates a new key for each column, this can be ignored.

To argue even more for why CTR is the correct choice: given a column size in the Parquet file, there are multiple row chunks, which individually are encrypted. By knowing the column chunk size and the column size, the OSWS could identify the offset within the range query a client might have sent, only fetching the range from S3 and only decrypting that part, essentially removing a big part of the overhead OSWS currently experiences.

### a) Parallelization:

When encrypting/decrypting the Parquet file, OSWS currently copies each row group sequentially. This should be an “embarrassingly parallelisable” operation, i.e. it could spawn threads to copy a cutout of the row groups to the new Parquet file so the row groups are copied in parallel. This should reduce copying overhead drastically, though in light of Section VII.B, it should be a write in place instead of a copy.

## C. Compression

For Parquet Sharp to copy over a column, it has to decompress the file. Here, OSWS thus modifies the file size and just keeps the file decompressed afterwards, when put into S3. It could have compressed it again, but it would not have fixed range queries due to modified metadata, etc. Here, with the custom reader and writer, it is not even needed to decompress the file. OSWS could potentially just keep the current format and still encrypt, etc., on a compressed or non-compressed file and thus leave all the sizes untouched.

## D. Encrypted Parquet Storage Cache

When Parquet files are fetched from S3, they are stored in local storage in their encrypted format. But as seen in Section VI.B.b, it has negligible performance improvements on the smaller files, while more significant improvements on larger Parquet files. This improvement might even become smaller and less useful with proper range support, as often-times only ranges of data are required and not the full file, and will thus be one of the later improvements, which might be tested on a future OSWS system. Of course, putting the caching in their unencrypted state and in DRAM cache could enhance performance as well, but would raise issues with handling access control on columns and size issues of the Parquet files, ending up filling up DRAM too quickly.

## E. Correct Decisions & OSWS Future

Even though OSWS has a lot of design choices, in light of prior subsections, a lot of correct decisions are believed

<sup>8</sup>Defined within [39]

to have been implemented, and are believed to need to be carried over to a V2 of a similar system.

a) *Envelope Encryption:*

Envelope encryption should still be used, even though it carries an initial significant overhead, but it is partly solved by having the decrypted DEK cache. This will also enable easier key rotation for future security improvements. A potential alternative would be to implement a fully fledged key vault inside OSWS, eliminating envelope encryption, though this would require many security measures.

b) *DEK Cache:*

The decrypted DEK cache is another one of these design choices which has to be carried over to a new version, given that a KV is used, as seen by Section VI.B.b and Section VI.B.a, that unwrapping DEKs carries a significant overhead.

c) *RBAC Metadata Storage:*

The microbenchmarks in Section VI.B.a also showed that the impact of authorizing users using the RBAC database was insignificant compared to the overhead of cryptographic operations and network transfer in general. This shows that a PostgreSQL database with the implemented design is a good option for RBAC Metadata Storage in the future.

d) *Role Management:*

Role management (creating and assigning roles, granting permissions) using the implemented “query editor” provided an intuitive and familiar way of managing roles. Though this is just a wrapper over the admin API, and it could be implemented in many ways, this “query editor” is a valid option that could be used in the future.

e) *Known Security Considerations:*

Two implementation-level security issues are acknowledged in the current OSWS release, within `KNOWN_ISSUES.md`.

First, S3 credential secret keys are stored in plaintext in the PostgreSQL database, as the SigV4 authentication handler requires the raw key for HMAC derivation. If the database is compromised, all S3 credentials become immediately usable. A mitigation would be to encrypt secret keys at rest using Azure Key Vault, decrypting on read during SigV4 verification.

Second, the RBAC admin flag is provisioned just-in-time from OIDC claims on every login. If the OIDC provider exposes this as a self-service field, users could grant themselves admin access. This is dependent on the OIDC provider configuration and can be mitigated by using a dedicated admin identity provider or restricting claim sources.

## VIII. CONCLUSION

The current implementation of OSWS proves that fine-grained role-based access control is possible within data lakes, and that query engines which do not cache sizing data can use it without any modifications, and that, depending on whether the “fully managed all-in-one cloud platforms” use it or not, the only change needed is to whitelist a third-party link to OSWS.

The PoC OSWS demonstrates the possibility of adding fine-grained role-based access control in data lakes by using

encryption with native Parquet files, and thus enforcing it within the Object Store. Query engines that do not cache Parquet sizing metadata, such as DuckDB, can use OSWS without modifications by pointing their S3 URL to OSWS.

The benchmarks showed that the DEK cache implemented in the OSWS was a completely necessary optimization. Without it, cold GET latency for a 5MB file is ~3.7 seconds. With the DEK cache enabled, the same operations take 92ms. But as soon as larger files were used, the latency quickly became large enough to make the current state of OSWS usable in production. These latencies were found to be the fault of two major parts of the system, and another one which has not been measured.

1st one was the latency given the KV. For this part, nothing much could be done by OSWS to improve the cold latency, though it was seen that using aggressive caching helped a lot. For cold improvements, the KV would have to be switched out with a more performant alternative.

2nd was the decryption. As Parquet Sharp was used within OSWS, some limitations were hit, such as the fact that the decryption had to be done sequentially, but also that when decryption happens, it actually has to copy the new data into a new Parquet file. This is extremely inefficient if a client only has access to a single column, and the result is that the whole file still has to be written.

The last thing, which was known, is that OSWS internally does not support ranged requests, as OSWS does not save metadata related to the byte-ranges nor the wrapped DEKs. This means that when a client requests a narrow range of information, OSWS has to fetch the whole file, and then it can find out which DEKs are needed in their unwrapped format. It can then copy over all the data to reflect the actual Parquet file in its correct size, as it does not copy over the DEK metadata. Now it can return the range to the client. This is an extreme amount of operations compared to the minuscule amount of data the client might be requesting, but given the design choices made, this has to be done this way.

These things limit the use-case of OSWS, and can more be used as learnings for created a future OSWS which solves these issues. Because in the current state OSWS has only solved objectives one and three to some extend, being making it support fine-grained access control by encryption in data lakes with RBAC and then the partly supporting 3rd party query engines. Objective two was completely missed being supporting “fully managed all-in-one cloud platforms”.

As discussed in Section VII some design changes would be able to solve these issues.

### A. Future Work

First the cryptographic metadata has to have its location changed from being in the Parquet metadata to being part of OSWS’s internal store.

The design changes needed are to implement a custom reader/writer.

The improvement with the most impact would be the custom Parquet reader and writer, supporting in-place encryption and decryption, using AES-CTR. This would eliminate sizing changes and enable proper range requests, given that

metadata is also moved, but also remove the need to copy over the whole Parquet file. The metadata specifically should be handled internally, and the Parquet footer will not be used to store extra metadata. Instead, the RBAC database will be extended further to store which DEKs are linked to which columns and those linked to the internal roles.

Additional future work includes: extending the S3-API coverage, implementing concurrency in row-group processing, and supporting compression pass-through to avoid decompression overhead without changing the size.

## I. APPENDIX

### A. Encrypted Parquet Footer

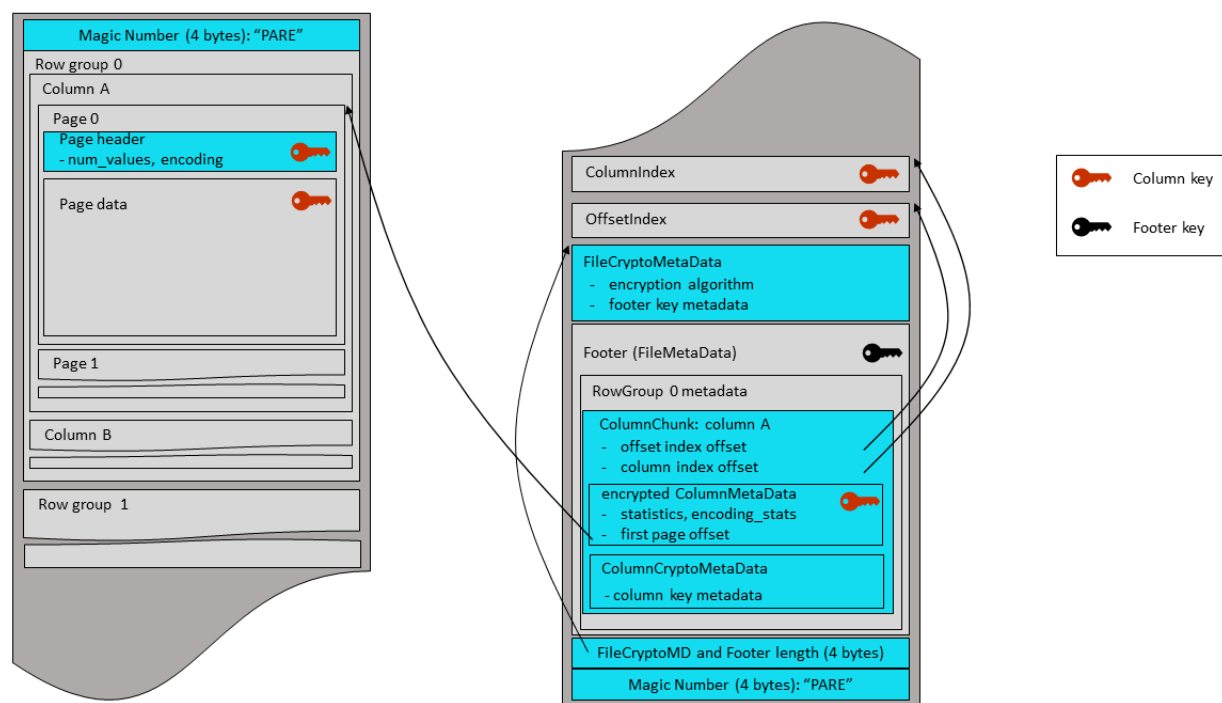


Fig. 1. Illustration of the Parquet file structure using an Encrypted Footer. From [18].



## B. VM Specification

```
root@ubuntu-s-1vcpu-1gb-35gb-intel-fral:~# inxi -Fxz
System:
  Kernel: 6.8.0-111-generic arch: x86_64 bits: 64 compiler: gcc v: 13.3.0
  Console: pty pts/6 Distro: Ubuntu 24.04.4 LTS (Noble Numbat)
Machine:
  Type: Kvm Mobo: DigitalOcean model: Droplet v: 20171212 serial: <filter> BIOS:
  DigitalOcean
  v: 20171212 date: 12/12/2017
CPU:
  Info: 8-core model: DO-Premium-AMD bits: 64 type: MCP arch: Zen 2 rev: 0 cache: L1:
  512 KiB
  L2: 4 MiB L3: 16 MiB
  Speed (MHz): avg: 1996 min/max: N/A cores: 1: 1996 2: 1996 3: 1996 4: 1996 5: 1996
  6: 1996
  7: 1996 8: 1996 bogomips: 31940
  Flags: avx avx2 ht lm nx pae sse sse2 sse3 sse4_1 sse4_2 sse4a ssse3 svm
Graphics:
  Device-1: Red Hat Virtio 1.0 GPU driver: virtio-pci v: 1 bus-ID: 00:02.0
  Display: server: No display server data found. Headless machine? tty: 110x29
  resolution: 1024x768
  API: EGL v: 1.5 drivers: kms_swrast,swrast platforms: active:
  gbm,surfaceless,device
  inactive: wayland,x11
  API: OpenGL v: 4.5 vendor: mesa v: 25.2.8-0ubuntu0.24.04.1 note: console (EGL
  sourced)
  renderer: llvmpipe (LLVM 20.1.2 256 bits)
Audio:
  Message: No device data found.
  API: ALSA v: k6.8.0-111-generic status: inactive
Network:
  Device-1: Intel 82371AB/EB/MB PIIX4 ACPI vendor: Red Hat Qemu virtual machine
  type: network bridge driver: N/A port: N/A bus-ID: 00:01.3
  Device-2: Red Hat Virtio network driver: virtio-pci v: 1 port: clc0 bus-ID: 00:03.0
  IF: eth0 state: up speed: -1 duplex: unknown mac: <filter>
  Device-3: Red Hat Virtio network driver: virtio-pci v: 1 port: clc0 bus-ID: 00:04.0
  IF: eth1 state: up speed: -1 duplex: unknown mac: <filter>
  IF-ID-1: br-cf3f443f079d state: down mac: <filter>
  IF-ID-2: docker0 state: down mac: <filter>
Drives:
  Local Storage: total: 320 GiB used: 19.4 GiB (6.1%)
  ID-1: /dev/vda model: N/A size: 320 GiB
  ID-2: /dev/vdb model: N/A size: 488 KiB
Partition:
  ID-1: / size: 308.93 GiB used: 19.28 GiB (6.2%) fs: ext4 dev: /dev/vda1
  ID-2: /boot size: 880.4 MiB used: 113.9 MiB (12.9%) fs: ext4 dev: /dev/vda16
  ID-3: /boot/efi size: 104.3 MiB used: 6.1 MiB (5.9%) fs: vfat dev: /dev/vda15
Swap:
  Alert: No swap data was found.
Sensors:
  Src: lm-sensors+/sys Message: No sensor data found using /sys/class/hwmon or lm-
  sensors.
Info:
  Memory: total: 16 GiB available: 15.62 GiB used: 1.03 GiB (6.6%)
  Processes: 180 Uptime: 4d 1h 2m Init: systemd target: graphical (5)
  Packages: 813 Compilers: N/A Shell: Bash v: 5.2.21 inxi: 3.3.34
```

Listing 3. VM specification used for the benchmarks of OSWS

### C. Micro Benchmark Results

| Benchmark: Decryption          |    |        |         |        |        |        |        |        |
|--------------------------------|----|--------|---------|--------|--------|--------|--------|--------|
| parameters                     | n  | mean   | stddev  | min    | p50    | p95    | p99    | max    |
| size=large                     | 15 | 7.48s  | 313.4ms | 7.12s  | 7.43s  | 8.17s  | 8.17s  | 8.17s  |
| size=medium                    | 15 | 1.68s  | 64.1ms  | 1.58s  | 1.67s  | 1.80s  | 1.80s  | 1.80s  |
| size=small                     | 15 | 65.7ms | 4.8ms   | 58.7ms | 64.2ms | 75.0ms | 75.0ms | 75.0ms |
| size=tiny                      | 15 | 13.6ms | 1.2ms   | 12.0ms | 13.4ms | 17.1ms | 17.1ms | 17.1ms |
| size=xlarge                    | 15 | 14.7s  | 452.1ms | 14.1s  | 14.6s  | 15.5s  | 15.5s  | 15.5s  |
| Benchmark: KeyUnwrap           |    |        |         |        |        |        |        |        |
| parameters                     | n  | mean   | stddev  | min    | p50    | p95    | p99    | max    |
| size=small,dek_bits=128        | 15 | 4.15s  | 166.5ms | 3.94s  | 4.15s  | 4.55s  | 4.55s  | 4.55s  |
| size=small,dek_bits=192        | 15 | 4.05s  | 198.0ms | 3.76s  | 3.99s  | 4.37s  | 4.37s  | 4.37s  |
| size=small,dek_bits=256        | 15 | 4.05s  | 186.2ms | 3.73s  | 4.04s  | 4.53s  | 4.53s  | 4.53s  |
| size=tiny,dek_bits=128         | 15 | 3.15s  | 152.0ms | 2.86s  | 3.12s  | 3.44s  | 3.44s  | 3.44s  |
| size=tiny,dek_bits=192         | 15 | 3.24s  | 214.5ms | 2.96s  | 3.22s  | 3.76s  | 3.76s  | 3.76s  |
| size=tiny,dek_bits=256         | 15 | 3.92s  | 113.8ms | 3.67s  | 3.97s  | 4.11s  | 4.11s  | 4.11s  |
| Benchmark: PermissionHierarchy |    |        |         |        |        |        |        |        |
| parameters                     | n  | mean   | stddev  | min    | p50    | p95    | p99    | max    |
| hierarchy_depth=0              | 15 | 4.3ms  | 0.7ms   | 3.5ms  | 4.2ms  | 5.8ms  | 5.8ms  | 5.8ms  |
| hierarchy_depth=16             | 15 | 6.8ms  | 2.3ms   | 5.1ms  | 6.2ms  | 14.8ms | 14.8ms | 14.8ms |
| hierarchy_depth=4              | 15 | 4.5ms  | 0.8ms   | 3.6ms  | 4.4ms  | 7.0ms  | 7.0ms  | 7.0ms  |
| hierarchy_depth=64             | 15 | 18.5ms | 2.0ms   | 14.7ms | 18.8ms | 23.6ms | 23.6ms | 23.6ms |
| Benchmark: PermissionService   |    |        |         |        |        |        |        |        |
| parameters                     | n  | mean   | stddev  | min    | p50    | p95    | p99    | max    |
| role_count=256                 | 15 | 24.7ms | 2.6ms   | 21.2ms | 24.3ms | 28.1ms | 28.1ms | 28.1ms |
| role_count=4                   | 15 | 6.7ms  | 4.8ms   | 4.3ms  | 5.3ms  | 24.6ms | 24.6ms | 24.6ms |
| role_count=64                  | 15 | 12.0ms | 1.4ms   | 9.6ms  | 12.2ms | 14.8ms | 14.8ms | 14.8ms |

Listing 4. Textual representation for the results of running the micro benchmarks

#### D. E2E Latency Benchmark Results

| PUT cache=n/a              |        |        |        |        |        |        |
|----------------------------|--------|--------|--------|--------|--------|--------|
| config                     | metric | tiny   | small  | medium | large  | xlarge |
| s3-direct                  | mean   | 27ms   | 69ms   | 575ms  | 2031ms | 4093ms |
|                            | stddev | 11ms   | 24ms   | 112ms  | 39ms   | 114ms  |
|                            | p50    | 24ms   | 60ms   | 535ms  | 2046ms | 4049ms |
|                            | p95    | 57ms   | 140ms  | 846ms  | 2076ms | 4317ms |
|                            | n      | 10     | 10     | 10     | 10     | 10     |
| osws-encrypt-cache         | mean   | 2208ms | 2024ms | 5024ms | 14.2s  | 26.5s  |
|                            | stddev | 660ms  | 94ms   | 161ms  | 354ms  | 466ms  |
|                            | p50    | 1994ms | 2034ms | 5046ms | 14.3s  | 26.5s  |
|                            | p95    | 4168ms | 2236ms | 5234ms | 14.7s  | 27.2s  |
|                            | n      | 10     | 10     | 10     | 10     | 10     |
| osws-encrypt-no-file-cache | mean   | 2150ms | 2090ms | 5228ms | 14.4s  | 26.1s  |
|                            | stddev | 584ms  | 81ms   | 139ms  | 353ms  | 557ms  |
|                            | p50    | 1965ms | 2081ms | 5236ms | 14.2s  | 26.2s  |
|                            | p95    | 3884ms | 2222ms | 5495ms | 15.2s  | 27.2s  |
|                            | n      | 10     | 10     | 10     | 10     | 10     |
| osws-encrypt-no-dek-cache  | mean   | 2558ms | 2327ms | 5152ms | 14.2s  | 25.3s  |
|                            | stddev | 567ms  | 264ms  | 207ms  | 197ms  | 772ms  |
|                            | p50    | 2383ms | 2263ms | 5054ms | 14.2s  | 25.4s  |
|                            | p95    | 4246ms | 2845ms | 5554ms | 14.6s  | 26.7s  |
|                            | n      | 10     | 10     | 10     | 10     | 10     |
| osws-no-encrypt            | mean   | 172ms  | 153ms  | 1940ms | 7456ms | 14.4s  |
|                            | stddev | 339ms  | 9ms    | 104ms  | 291ms  | 244ms  |
|                            | p50    | 48ms   | 151ms  | 1928ms | 7476ms | 14.4s  |
|                            | p95    | 1186ms | 172ms  | 2165ms | 7868ms | 14.9s  |
|                            | n      | 10     | 10     | 10     | 10     | 10     |

Listing 5. Textual representation of E2E Latency Benchmark results for PUT

GET cache=cold

| config                     | metric | tiny   | small  | medium | large  | xlarge |
|----------------------------|--------|--------|--------|--------|--------|--------|
| osws-encrypt-cache         | mean   | 3592ms | 3708ms | 6018ms | 13.1s  | 29.4s  |
|                            | stddev | 193ms  | 94ms   | 131ms  | 391ms  | 11.0s  |
|                            | p50    | 3558ms | 3703ms | 6024ms | 13.2s  | 22.6s  |
|                            | p95    | 3997ms | 3893ms | 6239ms | 13.8s  | 46.9s  |
|                            | n      | 10     | 10     | 10     | 10     | 10     |
| osws-encrypt-no-file-cache | mean   | 3662ms | 3819ms | 6436ms | 13.6s  | 29.8s  |
|                            | stddev | 116ms  | 160ms  | 283ms  | 248ms  | 11.3s  |
|                            | p50    | 3648ms | 3819ms | 6362ms | 13.6s  | 22.7s  |
|                            | p95    | 3863ms | 4166ms | 7007ms | 14.1s  | 47.2s  |
|                            | n      | 10     | 10     | 10     | 10     | 10     |
| osws-encrypt-no-dek-cache  | mean   | 20.1s  | 16.2s  | 79.6s  | 600.6s | 600.7s |
|                            | stddev | 797ms  | 811ms  | 3156ms | 369ms  | 387ms  |
|                            | p50    | 20.1s  | 15.9s  | 79.0s  | 600.5s | 600.9s |
|                            | p95    | 21.8s  | 18.3s  | 85.6s  | 601.2s | 601.1s |
|                            | n      | 10     | 10     | 10     | 10     | 10     |
| osws-no-encrypt            | mean   | 45ms   | 151ms  | 1303ms | 5017ms | 10.5s  |
|                            | stddev | 17ms   | 17ms   | 50ms   | 107ms  | 491ms  |
|                            | p50    | 38ms   | 153ms  | 1300ms | 5016ms | 10.2s  |
|                            | p95    | 92ms   | 177ms  | 1390ms | 5220ms | 11.6s  |
|                            | n      | 10     | 10     | 10     | 10     | 10     |

GET cache=warm

| config                     | metric | tiny   | small | medium | large  | xlarge |
|----------------------------|--------|--------|-------|--------|--------|--------|
| s3-direct                  | mean   | 12ms   | 33ms  | 477ms  | 1884ms | 3715ms |
|                            | stddev | 2ms    | 3ms   | 44ms   | 104ms  | 162ms  |
|                            | p50    | 11ms   | 33ms  | 490ms  | 1915ms | 3737ms |
|                            | p95    | 16ms   | 41ms  | 555ms  | 2015ms | 3987ms |
|                            | n      | 10     | 10    | 10     | 10     | 10     |
| osws-encrypt-cache         | mean   | 30ms   | 82ms  | 1600ms | 6040ms | 65.6s  |
|                            | stddev | 3ms    | 6ms   | 101ms  | 248ms  | 5962ms |
|                            | p50    | 29ms   | 82ms  | 1617ms | 6097ms | 62.9s  |
|                            | p95    | 37ms   | 92ms  | 1769ms | 6362ms | 75.0s  |
|                            | n      | 10     | 10    | 10     | 10     | 10     |
| osws-encrypt-no-file-cache | mean   | 35ms   | 130ms | 2298ms | 9090ms | 82.2s  |
|                            | stddev | 3ms    | 9ms   | 57ms   | 274ms  | 2451ms |
|                            | p50    | 34ms   | 129ms | 2282ms | 9025ms | 81.9s  |
|                            | p95    | 42ms   | 151ms | 2419ms | 9610ms | 87.8s  |
|                            | n      | 10     | 10    | 10     | 10     | 10     |
| osws-encrypt-no-dek-cache  | mean   | 16.0s  | 15.4s | 78.1s  | 508.5s | 600.4s |
|                            | stddev | 2087ms | 531ms | 4794ms | 140.2s | 291ms  |
|                            | p50    | 14.6s  | 15.4s | 76.5s  | 600.3s | 600.4s |
|                            | p95    | 20.3s  | 16.3s | 91.6s  | 600.8s | 601.0s |
|                            | n      | 10     | 10    | 10     | 10     | 10     |
| osws-no-encrypt            | mean   | 38ms   | 138ms | 1303ms | 5214ms | 9953ms |
|                            | stddev | 5ms    | 16ms  | 43ms   | 297ms  | 434ms  |
|                            | p50    | 36ms   | 133ms | 1302ms | 5103ms | 10.0s  |
|                            | p95    | 51ms   | 167ms | 1378ms | 5742ms | 10.6s  |
|                            | n      | 10     | 10    | 10     | 10     | 10     |

Listing 6. Textual representation of E2E Latency Benchmark results for GET with warm and cold cache

## REFERENCES

- [1] A. S. H. Trøstrup and L. F. T. Hanson, "Reviewing options for fine-grained Role-Based Access Control in Data Lakes." [Online]. Available: <https://github.com/lucasfth/reviewing-options-for-fine-grained-role-based-access-control-in-data-lakes>
- [2] S. Kumar, S. Yagati, C. Power, D. E. Culler, and R. A. Popa, "Membrane: A Cryptographic Access Control System for Data Lakes," *ArXiv*, 2025, [Online]. Available: <https://api.semanticscholar.org/CorpusID:281243750>
- [3] X. Shang *et al.*, "One Stone, Three Birds: Finer-Grained Encryption with Apache Parquet @ Large Scale," in *2022 IEEE International Conference on Big Data (Big Data)*, Osaka, Japan: IEEE, Dec. 2022, pp. 5802–5811. doi: [10.1109/BigData55660.2022.10020987](https://doi.org/10.1109/BigData55660.2022.10020987).
- [4] A. Jonker and M. M. Kosinski, "What is a data lake?." [Online]. Available: <https://www.ibm.com/think/topics/data-lake>
- [5] Deepstrike, [Online]. Available: <https://deepstrike.io/blog/what-is-jit-provisioning>
- [6] D. F. Ferraiolo and R. Kuhn, "Role-Based Access Controls," in *Proceedings of the 15th National Computer Security Conference (NCSC)*, Baltimore, MD, USA, Oct. 1992, pp. 554–563.
- [7] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, Aug. 2001, doi: [10.1145/501978.501980](https://doi.org/10.1145/501978.501980).
- [8] O. Foundation, "What is OpenID Connect." [Online]. Available: <https://openid.net/developers/how-connect-works/>
- [9] Microsoft, "What is Microsoft Entra?." [Online]. Available: <https://learn.microsoft.com/da-dk/entra/fundamentals/what-is-entra>
- [10] PocketID, "Introduction to PocketID." [Online]. Available: <https://pocket-id.org/docs/introduction>
- [11] AWS, "AWS Key Management Service." [Online]. Available: <https://docs.aws.amazon.com/kms/latest/developerguide/overview.html>
- [12] Azure, "Azure Key Vault Overview - Azure Key Vault." [Online]. Available: <https://docs.azure.cn/en-us/key-vault/general/overview>
- [13] AWS, [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcom.html>
- [14] A. S. Foundation, "Apache Spark." [Online]. Available: <https://spark.apache.org/>
- [15] M. A. Zaharia, A. Ghodsi, R. Xin, and M. Armbrust, "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," in *Conference on Innovative Data Systems Research*, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:229576171>
- [16] Apache, "Parquet File Format." [Online]. Available: <https://parquet.apache.org/docs/file-format/>
- [17] A. S. Foundation, "Apache Thrift." [Online]. Available: <https://thrift.apache.org/>
- [18] Apache, [Online]. Available: <https://parquet.apache.org/docs/file-format/data-pages/encryption/>
- [19] Google, [Online]. Available: <https://docs.cloud.google.com/kms/docs/envelope-encryption>
- [20] Redis, "TTL." [Online]. Available: <https://redis.io/docs/latest/commands/ttl/>
- [21] E. Barker, *Recommendation for Key Management Part 1: General*, no. NIST SP 800-57pt1r4. 2016, p. NIST SP 800-57pt1r4. doi: [10.6028/NIST.SP.800-57pt1r4](https://doi.org/10.6028/NIST.SP.800-57pt1r4).
- [22] Lakekeeper, "Lakekeeper Docs - Storage." [Online]. Available: <https://docs.lakekeeper.io/docs/0.5.x/storage/None>
- [23] Apache, [Online]. Available: <https://polaris.apache.org/releases/1.4.1/>
- [24] R. Anderson and T. Dykstra, "Tutorial: Create a Minimal API with ASP.NET Core." [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/tutorials/min-web-api?view=aspnetcore-10.0>
- [25] msmbaldwin, "Azure data encryption at rest - Azure Security." [Online]. Available: <https://learn.microsoft.com/en-us/azure/security/fundamentals/encryption-atrest>
- [26] "Authenticating Requests (AWS Signature Version 4)." [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/API/sig-v4-authenticating-requests.html>
- [27] [Online]. Available: <https://bitwarden.com/resources/passkeys-are-phishing-resistant-mfa/>
- [28] shadcn, "Introduction to shadcn." [Online]. Available: <https://ui.shadcn.com/docs>
- [29] peggyjs, "peggyjs/peggy – Parser Generator for JavaScript." [Online]. Available: <https://github.com/peggyjs/peggy>
- [30] [Online]. Available: <https://github.com/G-Research/ParquetSharp>
- [31] "Reading and writing Parquet files." [Online]. Available: <https://arrow.apache.org/docs/cpp/parquet.html>
- [32] Snowflake, "Introduction to external tables." [Online]. Available: [https://docs.snowflake.com/en/user-guide/tables-external-intro?utm\\_source=DV-Blog%3Fsource&utm\\_cta=website-workload-ai-ml-timely-content-building-a-chatbot-app-with-rag-and-feature-store](https://docs.snowflake.com/en/user-guide/tables-external-intro?utm_source=DV-Blog%3Fsource&utm_cta=website-workload-ai-ml-timely-content-building-a-chatbot-app-with-rag-and-feature-store)
- [33] Snowflake, "External table files." [Online]. Available: [https://docs.snowflake.com/en/sql-reference/functions/external\\_table\\_files](https://docs.snowflake.com/en/sql-reference/functions/external_table_files)
- [34] ClickHouse, "ClickHouse Example Datasets." [Online]. Available: <https://clickhouse.com/docs/getting-started/example-datasets>
- [35] T. W. Schneider, "Analyzing 1.1 Billion NYC Taxi and Uber Trips, with a Vengeance." [Online]. Available: <https://toddwtschneider.com/posts/analyzing-1-1-billion-nyc-taxi-and-uber-trips-with-a-vengeance/>
- [36] "Companies using MinIO in 2026 | Landbase." Accessed: Mar. 23, 2026. [Online]. Available: <https://data.landbase.com/technology/minio/>
- [37] "Companies that use Minio (2,450)." Accessed: Mar. 23, 2026. [Online]. Available: <https://theirstack.com/en/technology/minio>
- [38] DuckDB Foundation, "ducklake\_data\_file." [Online]. Available: [https://ducklake.select/docs/stable/specification/tables/ducklake\\_data\\_file.html](https://ducklake.select/docs/stable/specification/tables/ducklake_data_file.html)
- [39] "Parquet Modular Encryption." [Online]. Available: <https://parquet.apache.org/docs/file-format/data-pages/encryption/>
- [40] H. Lipmaa, P. Rogaway, and D. Wagner, "Comments to NIST concerning AES-modes of operations: CTR-mode encryption," p. , 2000.